

The Traditional Approach to Requirements

CHAPTER B

CHAPTER OUTLINE

- Traditional and Object-Oriented Views of Activities and Use Cases
- Data Flow Diagrams
- Documentation of DFD Components
- Locations and Communication through Networks

LEARNING OBJECTIVES

After reading this chapter, you should be able to:

- Explain how the traditional approach and the object-oriented approach differ when modeling the details of a use case
- List the components of a traditional system and the symbols representing them on a data flow diagram
- Describe how data flow diagrams can show the system at various levels of abstraction
- Develop data flow diagrams, data element definitions, data store definitions, and process descriptions
- Develop tables to show the distribution of processing and data access across system locations

OPENING CASE SAN DIEGO PERIODICALS: FOLLOWING THE DATA FLOW

Arturo Romero and Lei Xu were meeting to review the first draft of several data flow diagrams for San Diego Periodicals' new advertising billing system. Arturo was the analyst assigned to define the new system requirements. Lei was the manager in charge of advertising accounts; she knows virtually all there is to know about how the current system operates. The two had met several times before. Their most recent meeting (just last week) reviewed details of the current system events, ad request processing, and process participants. Arturo left that meeting with pages of notes and sample forms and reports from the current system. Arturo had telephoned Lei several times since that meeting to ask additional questions.

Arturo began the review by saying, "The materials and information that you gave me last week took me quite awhile to absorb, but I think I was able to understand and document all the system activities or use cases that we discussed. The purpose of this meeting is to ensure that the processing requirements I've written down are complete and accurate. Let's start with a few of the diagrams that I've created."

Arturo laid out three diagrams on the table. Lei looked at them briefly and said, "I've never seen diagrams like this before; they look like blueprints for playing a game of marbles. And I thought the entity-relationship diagrams were strange!"

Arturo replied, "I expect this review will go slowly because this is your first look at this style of documentation. I'll explain how to interpret the diagrams as we go along. Ask as many questions as you like. The quality of our work depends on your understanding the diagrams, so don't be shy."

Arturo continued, "The pictures are called data flow diagrams, or DFDs for short. They divide your system

into processing functions, represented by the rectangles with rounded-off corners. The arrows show data movement among processes and between processes and files." Lei pointed to a square on one of the diagrams and said, "I assume that this is a company purchasing ad space?"

Arturo replied, "Yes, the squares represent people or organizations that supply inputs or expect output data from the system."

Lei said, "I think I can get the hang of this. I recognize most of the names that you've used for the processes and data. I'm not sure what these other symbols are; they're named for things we store in our manual files and database, but they don't seem to correspond exactly to our system."

"They don't," Arturo replied. "They're entities from the entity-relationship diagram that we developed a couple of weeks ago. But let's skip over those for the moment. Why don't we walk through the processing sequence for booking an ad, and we'll discuss the entities as we get to them?"

Arturo and Lei continued reviewing the DFDs, and the next thing they knew, over an hour had passed. Several pages of Arturo's notepad had been filled, and 25 corrections and comments were noted in red on the data flow diagrams. Lei said, "My brain feels completely drained. I don't think that I can do any more of this today."

Arturo replied, "You've given me plenty of things to work on, so let's call it quits for now. Can we meet for two hours at nine o'clock on Thursday?"

Lei replied, "Yes, I'm free then. So, will you be bringing more data flow diagrams or do you have something even weirder up your sleeve?"

Arturo smiled and said, "The toughest stuff is behind us, but you should expect a few more surprises."

■ Overview

Chapters 3, 4, and 5 described two key concepts associated with modeling system requirements in the traditional and the object-oriented approaches to information systems development: the use cases and the things involved in users' work. In Chapter 3, you learned to identify use cases and describe them with use case diagrams. In Chapter 4, you learned to describe things in the user's work domain by using class and entity-relationship diagrams. In Chapter 5, you learned how to describe use cases in further detail with use case descriptions, activity diagrams, sequence diagrams, and state-machine diagrams. By the end of Chapter 5, you had mastered all the object-oriented models used to describe a system's functional requirements.

This chapter describes an older and more traditional approach to representing requirements. In this approach, entity-relationship diagrams represent things in the user's work domain. However, the models that represent activities

(use cases) and the interaction among activities and things are very different from object-oriented models. Although object-oriented activity models are more widely used than traditional models, the traditional models are still used in many contexts, including documentation for older systems and to present requirements for new systems to users in an easy-to-read fashion. Thus, learning how to read and develop traditional models is a useful skill for the modern systems analyst.

■ Traditional and Object-Oriented Views of Activities and Use Cases

The traditional and object-oriented approaches to system development differ in how a system's response to an event is modeled and implemented. The traditional approach views a system as a collection of processes—some performed by people and some performed by computers. Traditional computer processes are much like procedural computer programs; they contain instructions that execute in a sequence. When the process executes, it interacts with stored data, reading data values and then writing other data values back to the data file. The process might also interact with people, such as when an instruction asks a user to input a value or it displays information to a user on the computer screen. The traditional approach to systems, then, involves processes, stored data, inputs, and outputs and it includes processing models that emphasize these system features, as shown on the left side of **Figure B-1**.

The object-oriented approach views a system as a collection of interacting objects, as summarized on the right side of Figure B-1. The objects are based on things in the problem domain, as discussed in Chapter 4. Objects are capable of behaviors (called *methods*) that allow them to interact with each other and with people using the system. One object asks another object to do something by sending it a message. There are no conventional computer processes, data files, or databases per se. Objects carry out the activities and remember the data values. When modeling what the system does in response to an event, the object-oriented approach includes models that show objects, their behavior, and their interactions with other objects.

Because of these differences summarized in Figure B-1, the traditional and object-oriented approaches to requirements employ different models, as summarized in **Figure B-2**. The models on the right side of the figure were described in Chapters 3, 4, and 5. Entity-relationship diagrams (ERDs) were described in Chapter 4. The remainder of this chapter explores the traditional models on the lower left side of **Figure B-2**.

■ Data Flow Diagrams

The traditional approach to information system development describes activities as processes carried out by people or computers. A graphical model that has proven to be quite valuable for modeling processes is the data flow diagram.

FIGURE B-1 *Traditional versus object-oriented approaches*

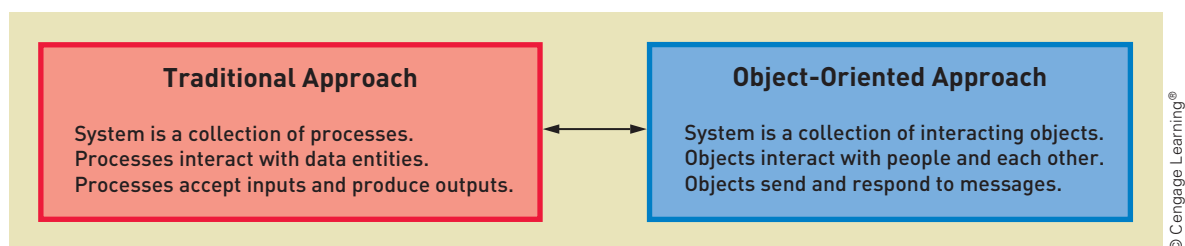
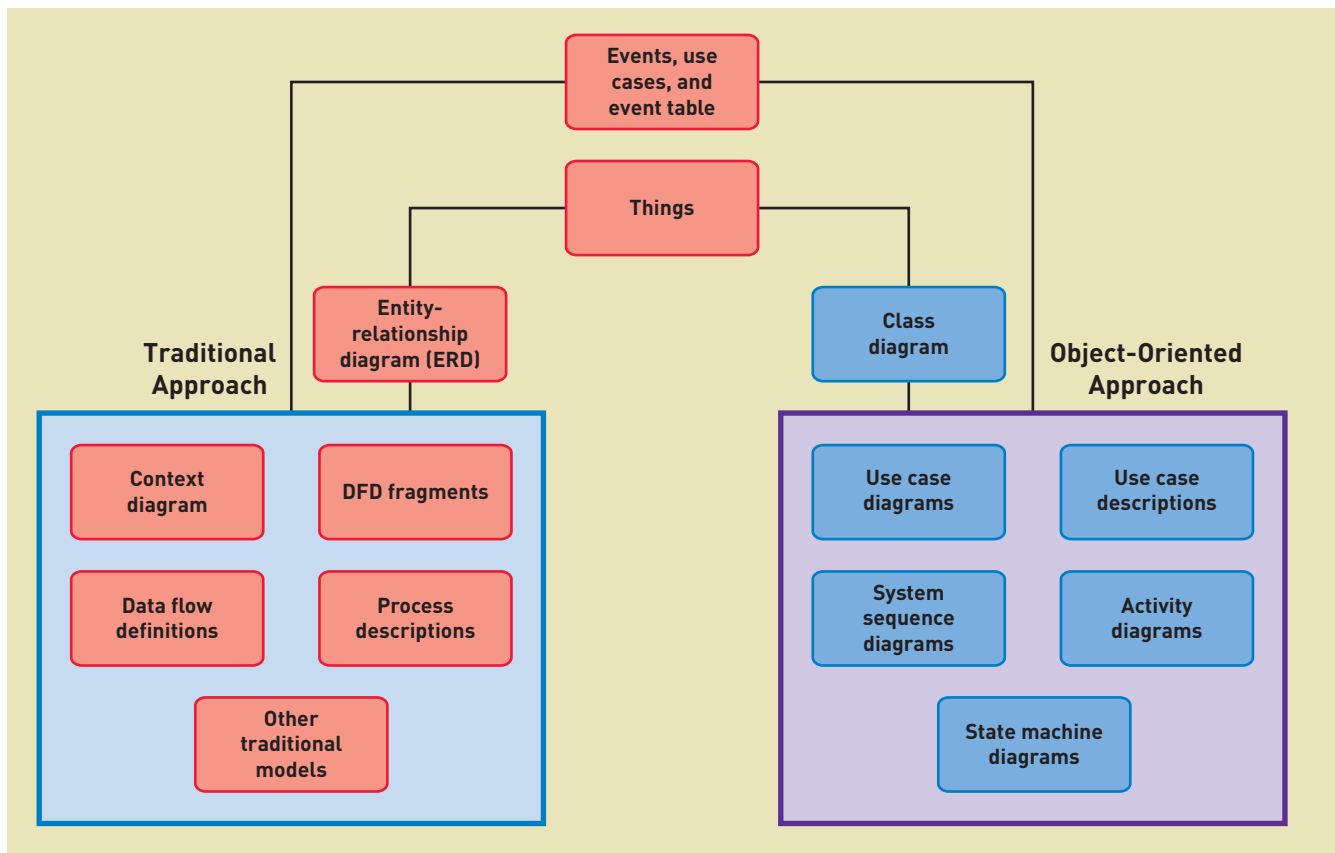


FIGURE B-2 Requirements models for the traditional and object-oriented approaches



© Cengage Learning®

data flow diagram (DFD) a diagram that represents system requirements as processes, external agents, data flows, and data stores

external agent a person or organization, outside the system boundary, that supplies data inputs or accepts data outputs

process a symbol on a DFD that represents an algorithm or procedure by which data inputs are transformed into data outputs

data flow an arrow on a DFD that represents data movement among processes, data stores, and external agents

data store a place where data is held pending future access by one or more processes

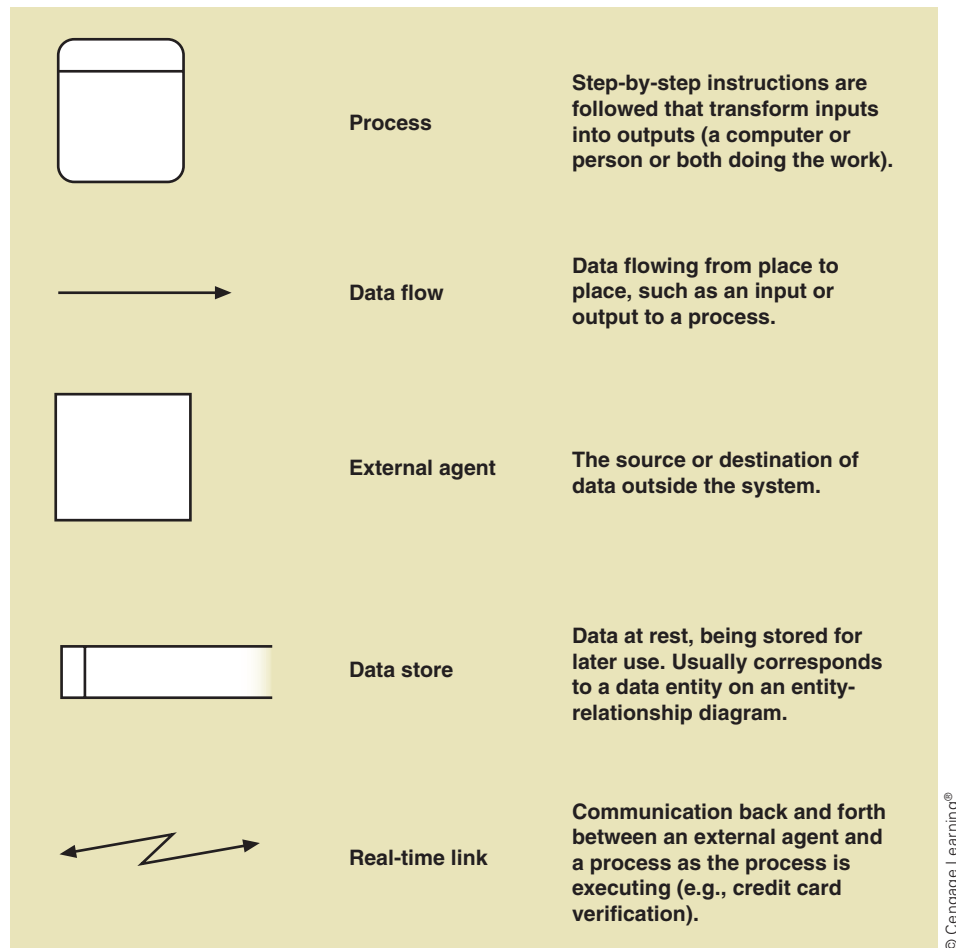
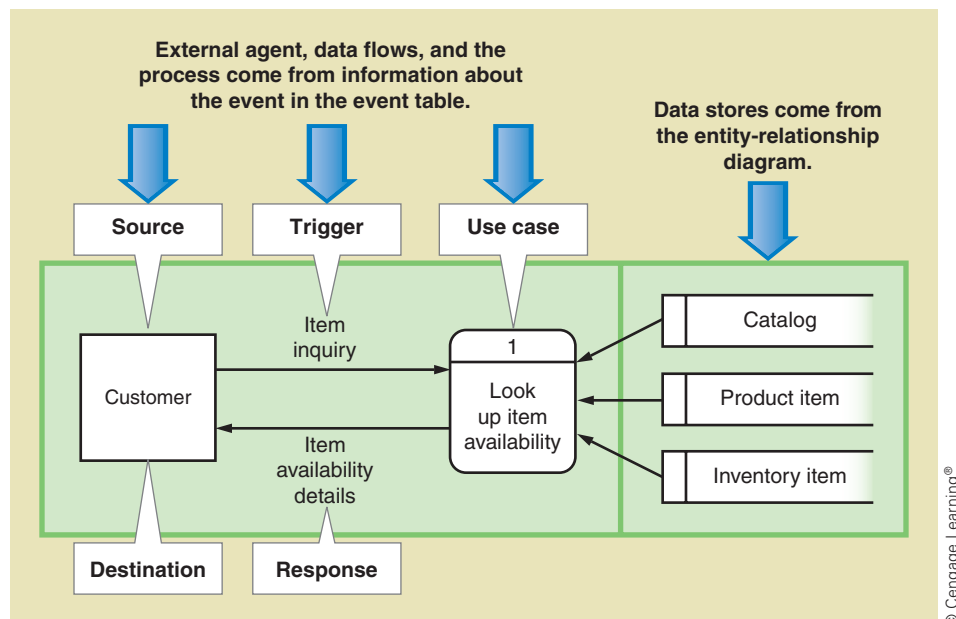
Activity diagrams are also used, but the data flow diagram is the most commonly used process model.

A **data flow diagram (DFD)** is a graphical system model that shows all the main requirements for an information system in one diagram: inputs and outputs, processes, and data storage. Everyone working on a development project can see all aspects of the system working together at once with the DFD. That is one reason for its popularity. The DFD is also easy to read because it is a graphical model and because there are only five symbols to learn (see [Figure B-3](#)). End users, management, and all information systems workers can typically read and interpret the DFD with minimal training.

[Figure B-4](#) shows an example of a data flow diagram representing a portion of the Ridgeline Mountain Outfitters (RMO) customer support system. Recall from Chapter 2 that the CSS supports catalog, telephone, and Web-based orders and is one of the systems that will be replaced by the new Consolidated Sales and Marketing System. The CSS was first deployed 15 years ago and incrementally updated four years later. Traditional and object-oriented analysis models were commonly used at that time. Thus, the CSS is documented by using a combination of ERDs, DFDs, and use case descriptions.

The square on the left side of [Figure B-4](#) is an **external agent**—Customer, the Source and Destination for some data outside the system. The rectangle with rounded corners is a **process** named Look up item availability that can also be referred to by its number (1). A process defines rules for transforming inputs to outputs. The lines with arrows are **data flows**. [Figure B-4](#) shows two data flows between Customer and process 1: a process input named Item inquiry and a process output named Item availability details. The final symbol—the flat, open-ended rectangle—is a **data store**. Each data store represents a file or part of a database that stores information about a data entity. In this example, data

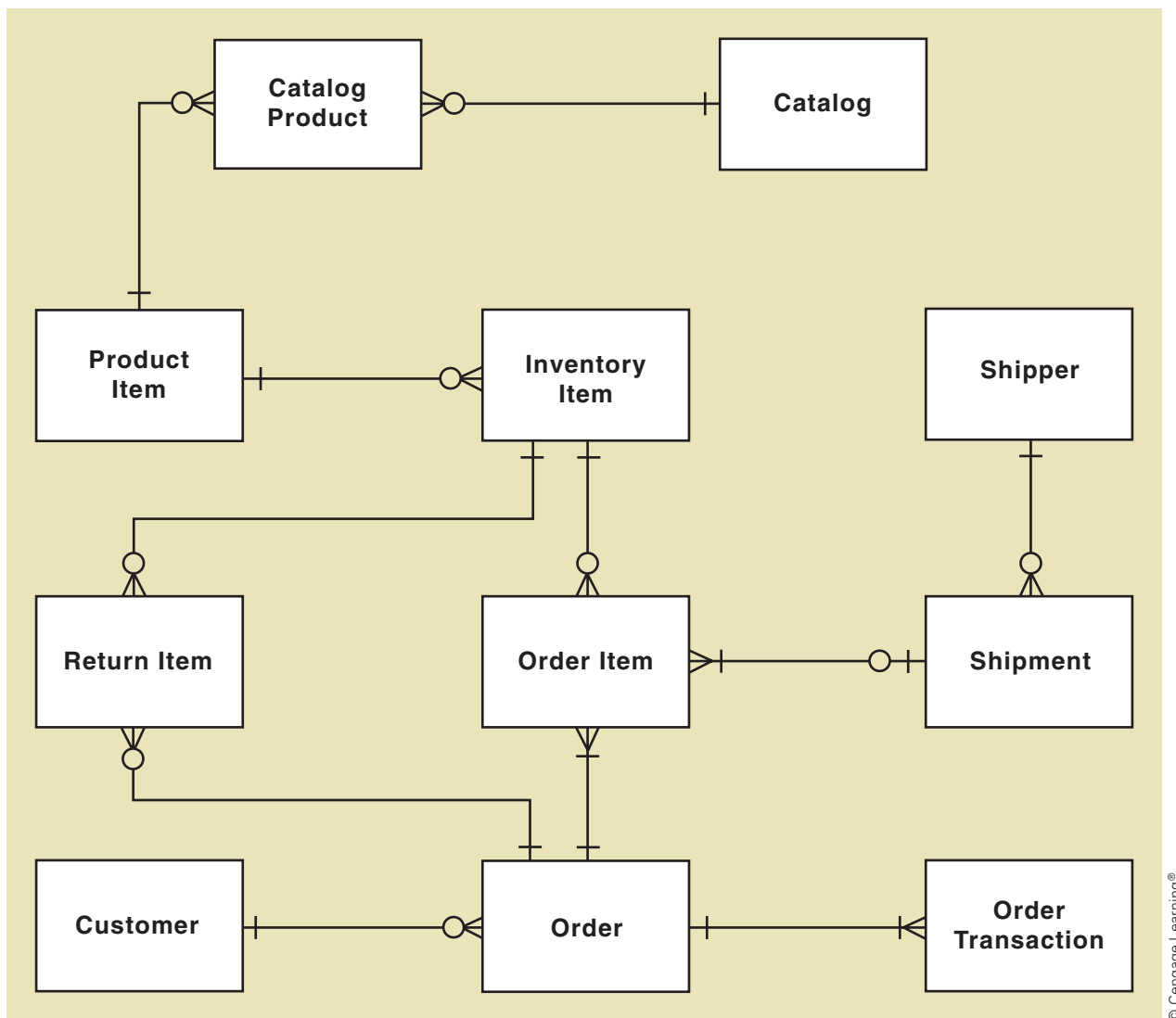
FIGURE B-3 Data flow diagram symbols

FIGURE B-4 A DFD showing the process *Look up item availability* (a DFD fragment from the RMO case)

flows (lines with arrows) point from the data stores to the process, meaning that the process looks up information in the data stores named *Catalog*, *Product item*, and *Inventory item*.

Process 1 in Figure B-4 corresponds to a use case that responds to the event *Customer wants to check item availability*. The use case trigger is a data flow

FIGURE B-5 The DFD integrates events, use cases, and the ERD



named Item inquiry, the source is the Customer, the response is a data flow named Item availability details, and the destination for the response is Customer. Each data store represents an entity from the CSS entity-relationship diagram (ERD) shown in **Figure B-5**. Therefore, the data flow diagram shows the system use case in response to this one event in graphical form and integrates processing triggered by the event with the data entities modeled using the ERD.

■ Data Flow Diagrams and Levels of Abstraction

Many different types of data flow diagrams are produced to show system requirements. The example just described is a DFD fragment, showing one process in response to one event. Other data flow diagrams show the processing at either a higher level (a more general view of the system) or at a lower level (a more detailed view of one process). These differing views of the system (high level versus low level) are called **levels of abstraction**.

Data flow diagrams can show either higher- or lower-level views of the system. The high-level processes on one DFD can be decomposed into separate lower-level, detailed DFDs. Processes on the detailed DFDs can also be decomposed into additional diagrams to provide multiple levels of abstraction.

levels of abstraction any modeling technique that breaks the system into a hierarchical set of increasingly more detailed models

FIGURE B-6 Layers of DFD abstraction for a course registration system

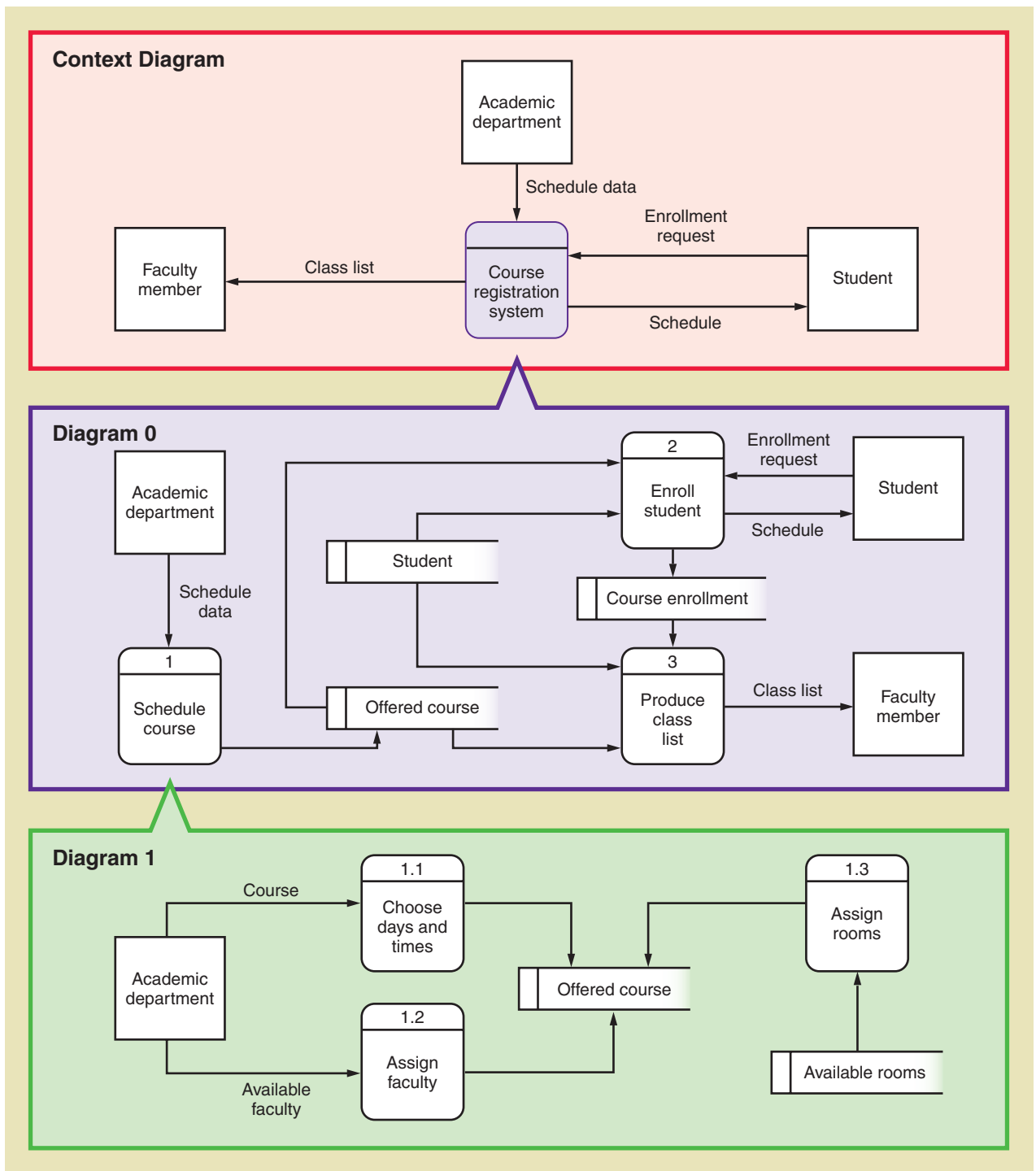


Figure B-6 shows how DFDs at each level of detail provide additional information about one process at the next higher level. The topmost DFD shows the most abstract representation of the course registration system as a single process. The middle DFD shows internal details of a context diagram process. The bottom DFD shows internal details of process 1 in the middle DFD. Each DFD abstraction level is described further in the following sections.

context diagram a DFD that summarizes all processing activity within the system in a single process symbol

■ Context Diagram

A **context diagram** is a DFD that describes the most abstract view of a system. All external agents and all data flows into and out of the system are shown in one diagram, with the entire system represented as one process. The top-most DFD in Figure B-6 is a context diagram for a simple university course registration system that interacts with three external agents: Academic department, Student, and Faculty member. Academic departments supply information on offered courses, students request enrollment in offered courses, and faculty members receive class lists when the registration period is complete.

A context diagram clearly shows the system boundary. The system scope is defined by what is represented within the single process and what is represented as external agents. External agents that supply or receive data from the system are outside the system scope, and everything else is inside the system scope. The context diagram doesn't usually show data stores because all the system's data stores are considered to be within the system scope (that is, part of the internal implementation of the process that represents the system). However, data stores may be shown when they are shared by the system being modeled and another system.

The context diagram is usually created in parallel with the initial list of use cases and events. Each trigger for an external event becomes an input data flow, and the source becomes an external agent. Each response becomes an output data flow, and the destination becomes an external agent. Triggers for temporal events aren't data flows, so there are no input data flows for temporal events.

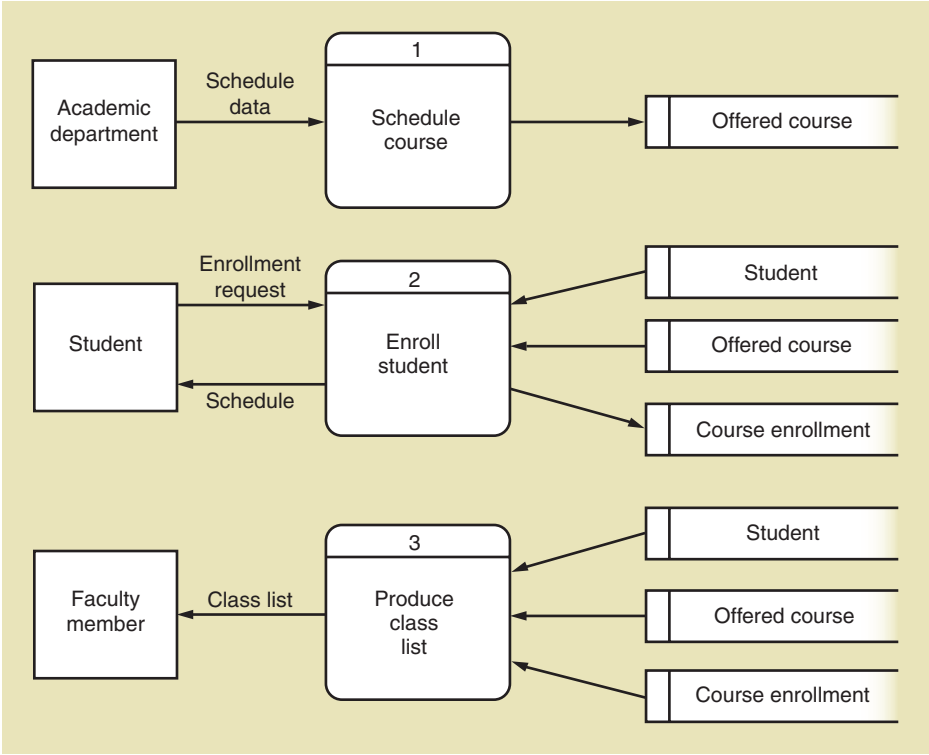
■ DFD Fragments

DFD fragment a DFD that represents the system response to one event within a single process symbol

A **DFD fragment** is created for each use case triggered by an event in the event table. Each DFD fragment is a self-contained model showing how the system responds to a single event. The analyst usually creates DFD fragments one at a time, focusing attention on each part of the system. The DFD fragments are drawn after the event table and context diagram are complete.

Figure B-7 shows the three DFD fragments for the simple course registration system. Each DFD fragment represents all processing for a use case

FIGURE B-7 Three DFD fragments for the course registration system



triggered by an event within a single process symbol. The fragments show details of interactions among the process, external agents, and internal data stores. The data stores used on a DFD fragment represent entities on the ERD. Each DFD fragment shows only those data stores that are actually needed to respond to the event.

■ The Event-Partitioned System Model

All the DFD fragments for a system or subsystem can be combined on a single DFD called the **event-partitioned system model (diagram 0)**. **Figure B-8** shows how the three course registration system DFD fragments shown in Figure B-7 are combined to create diagram 0.

event-partitioned system model (diagram 0) a DFD that models system requirements by using a single process for each event in a system or subsystem

FIGURE B-8 Combining DFD fragments to create the event-partitioned system model for the course registration system

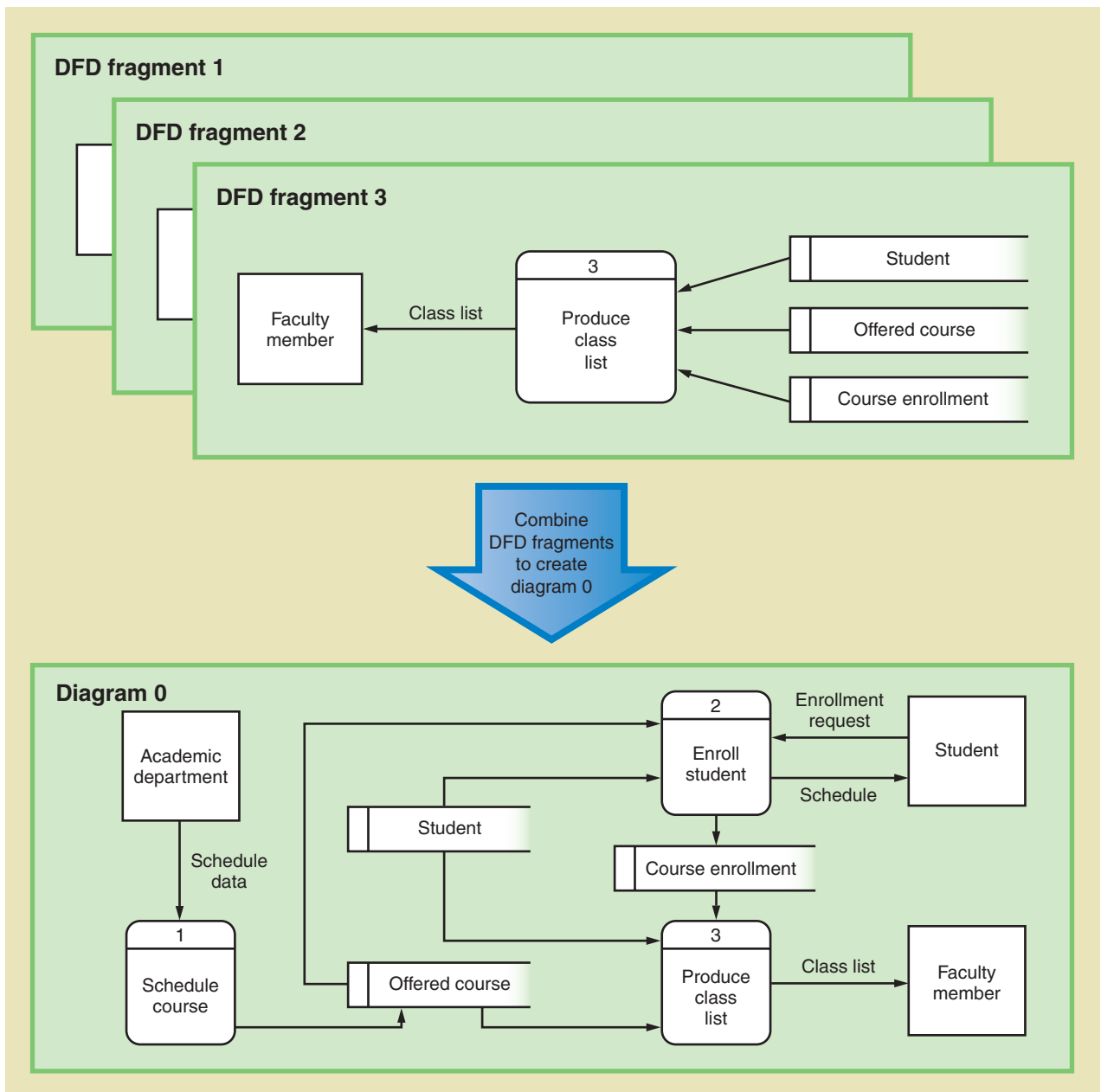


Diagram 0 is used primarily as a presentation tool. It summarizes an entire system or subsystem in greater detail than does a context diagram. However, analysts often avoid developing diagram 0 because:

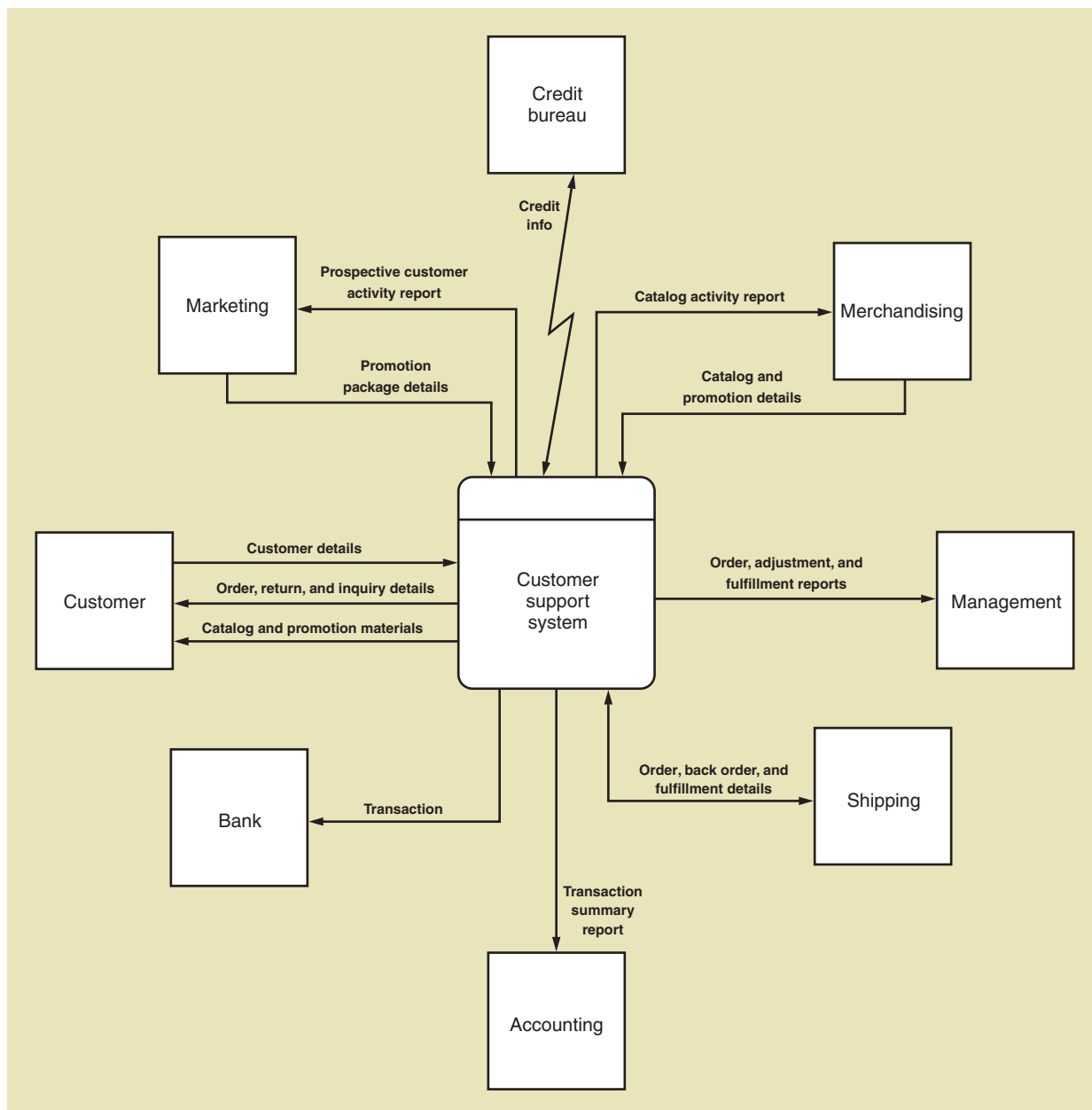
- The information content duplicates the set of DFD fragments.
- The diagram is often complex and unwieldy, particularly for large systems that respond to many events.

As we will discuss later in this chapter, redundancy and complexity are two DFD characteristics that analysts should avoid whenever possible.

■ RMO Data Flow Diagrams

Figure B-9 shows a context diagram for the existing RMO customer support system. Normally, data flows and external agents on the context diagram are

FIGURE B-9 A context diagram for the RMO customer support system



taken directly from the event table as discussed previously, but because the RMO customer support system responds to 20 events, this figure combines data flows for some events for simplicity. In a smaller system example with 10 to 15 events, you should include all data flows on the context diagram.

When a system responds to many events, it is commonly divided into subsystems, and a context diagram is created for each subsystem. **Figure B-10** divides the RMO customer support system into subsystems based on use case similarities, including interactions with external agents, interactions with data stores, and similarities in required processing. **Figure B-11** shows the context diagram for the order-entry subsystem.

FIGURE B-10 RMO subsystems and use cases for each subsystem

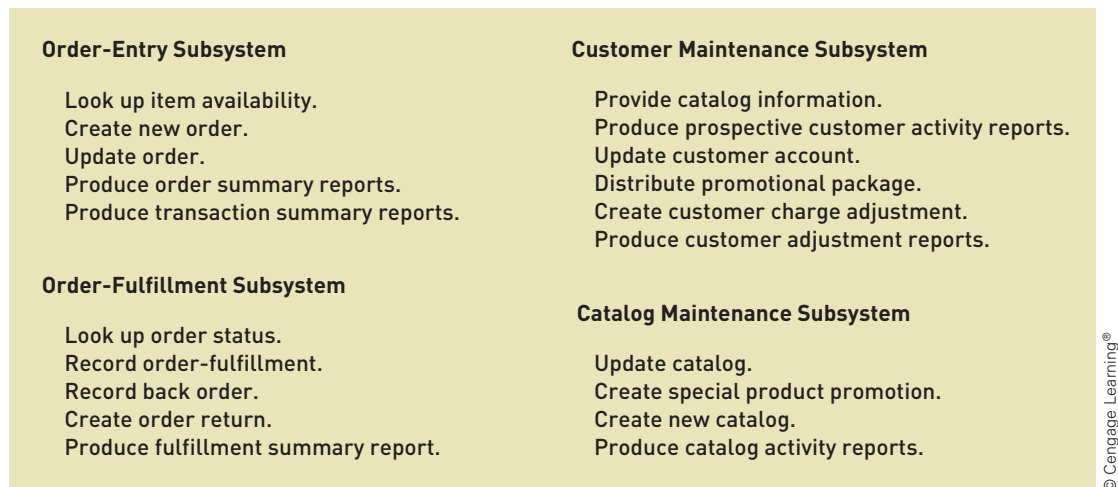


FIGURE B-11 A context diagram for the RMO order-entry subsystem

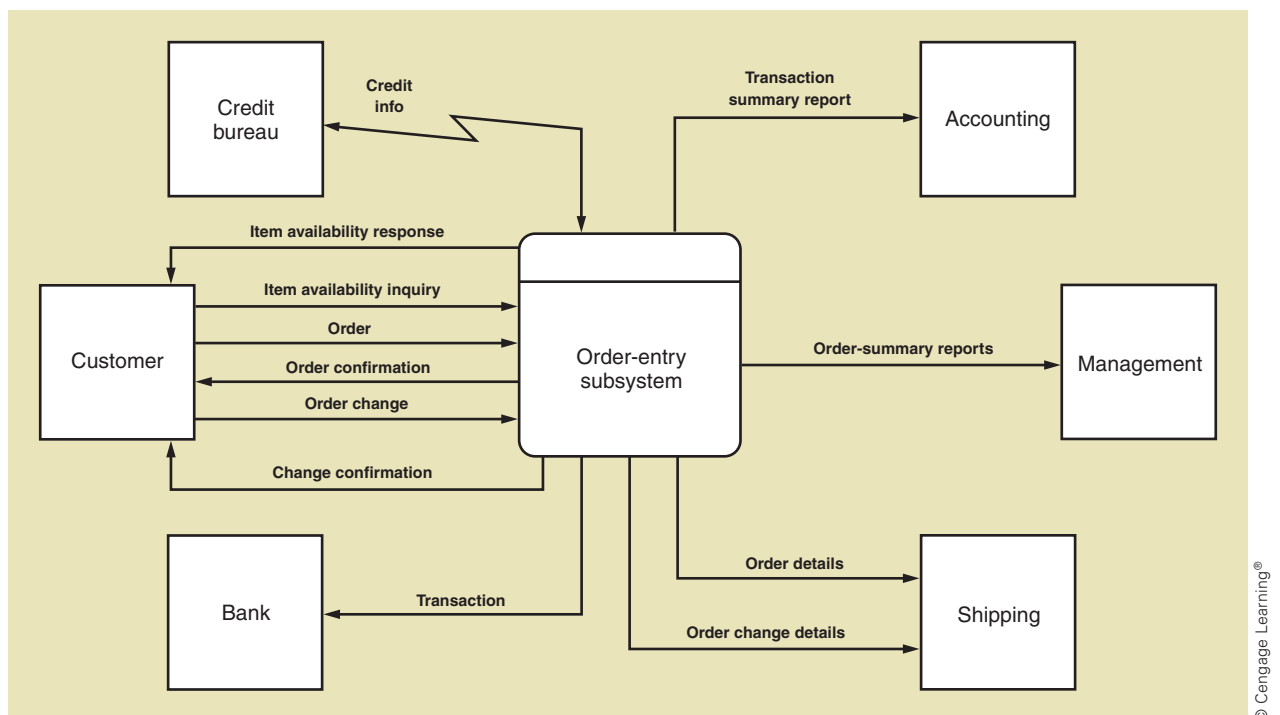
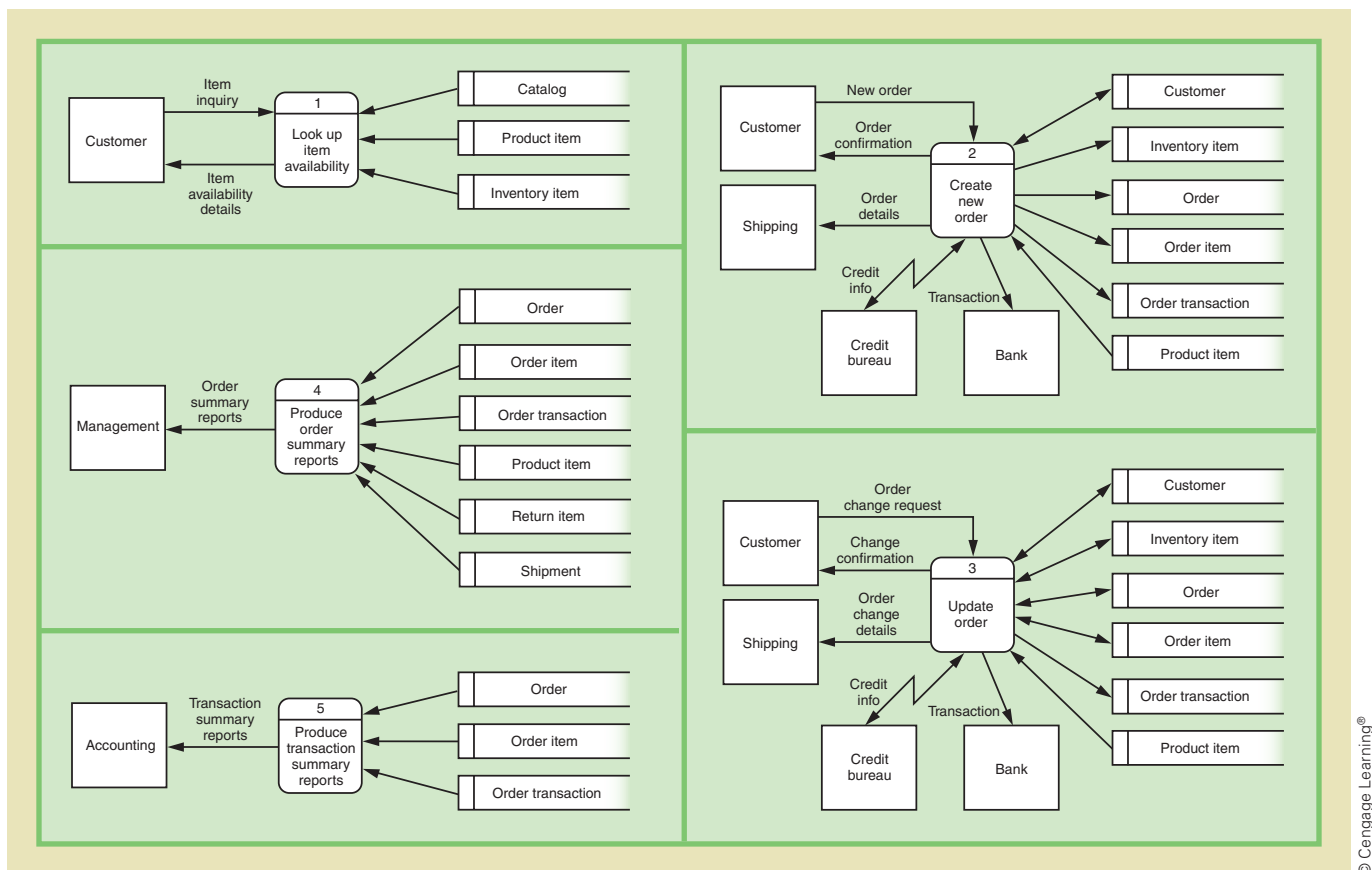


FIGURE B-12 DFD fragments for the RMO order-entry subsystem



© Cengage Learning®

Figure B-12 shows the DFD fragments for the RMO order-entry subsystem. Note that there are five DFD fragments—one for each order-entry subsystem use case listed in Figure B-10.

Similarly, **Figure B-13** shows the RMO order-entry subsystem diagram 0, the result of combining the DFD fragments from Figure B-12. To simplify the diagram and make it more readable, the seven data stores in Figure B-12 are collapsed into a single data store in Figure B-13. Recall that diagram 0 is just used as a presentation aid. The DFD fragments show which processes interact with which individual data stores.

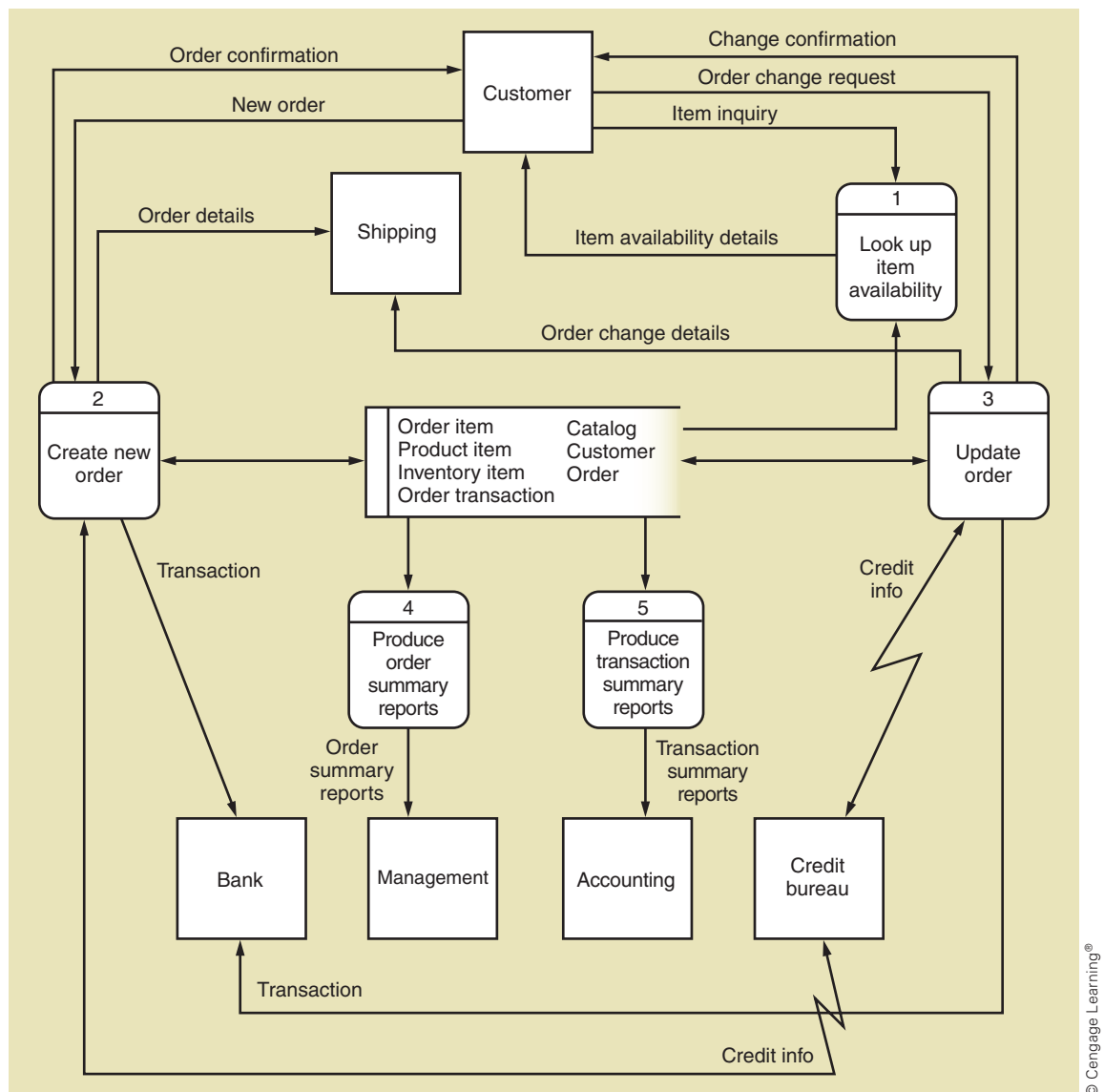
■ Decomposition to See One Activity's Detail

Some DFD fragments involve a lot of processing that the analyst needs to explore in more detail. As with any modeling step, further decomposition helps the analyst learn more about the requirements while also producing needed documentation. **Figure B-14** shows an example of a more detailed diagram for the CSS DFD fragment 2: Create new order. It is named diagram 2 because it shows the “insides” of process 2. The subprocesses are numbered 2.1, 2.2, 2.3, and 2.4. But the numbering system doesn't necessarily imply the sequence of subprocess execution.

The diagram decomposes process 2 into four subprocesses: Record customer information, Record order, Process order transaction, and Produce confirmation. These subprocesses are viewed as the four major steps required to complete the activity. This decomposition is just one way to divide the work. Another analyst might arrive at a different solution.

The first step begins when a customer provides the information making up the New order data flow. The New order data flow contains all the information about

FIGURE B-13 An event-partitioned model of the order-entry subsystem (diagram 0)

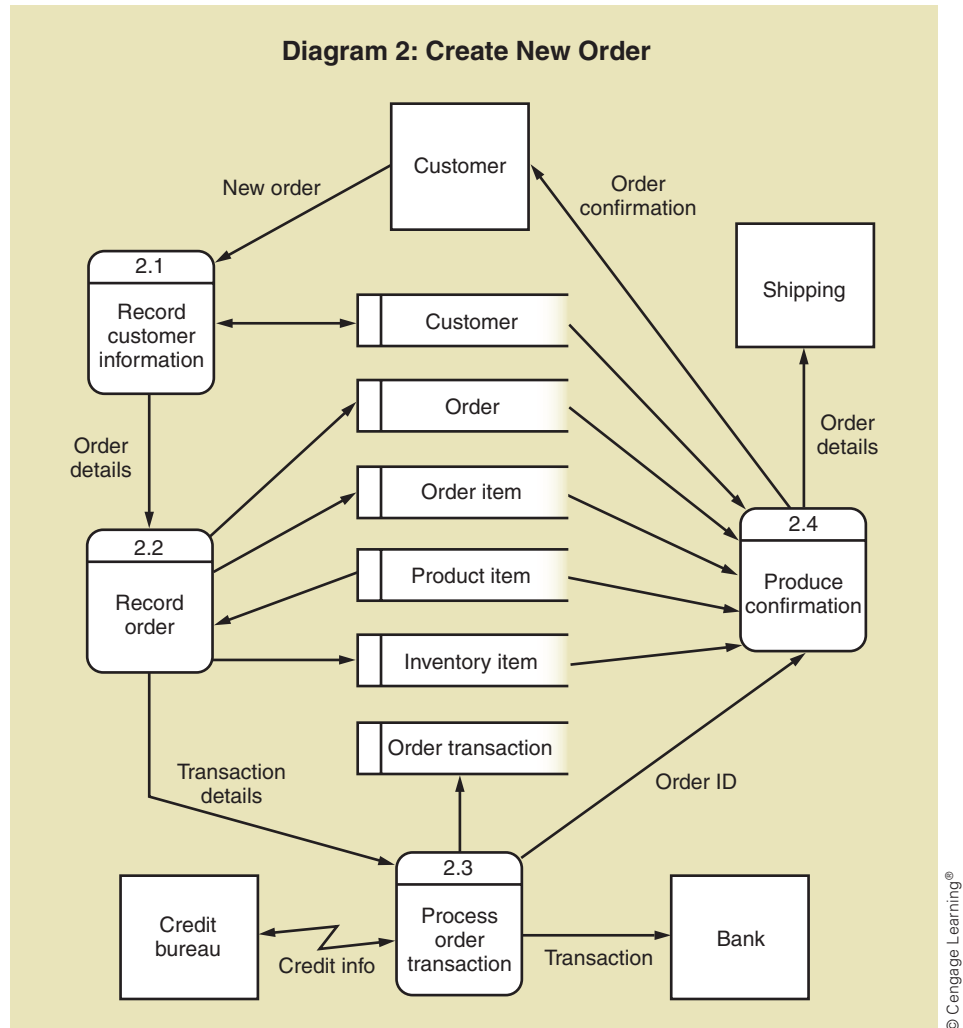


the customer and the items the customer wants to order. If the customer is new, process 2.1 stores the customer information in the data store named Customer (creating a new customer record or updating existing customer information as required). Remember that the data store represents the customer data entity on the ERD (see Figure B-5). Process 2.1 then sends the rest of the information about the order—a data flow named Order details—on to process 2.2.

Process 2.2 takes the Order details data flow and creates a new order record by adding data to the Order data store. Then, for each item ordered, the stock on hand and the current price are looked up in the Product item and Inventory item data stores. If adequate stock is on hand, an order item record is created for that item and the stock on hand for the inventory item record is changed. If three items are ordered, one order record is created and three order item records are created.

Process 2.2 adds up the total amount due for the order (price times quantity for each item) and sends the data flow named Transaction details to process 2.3 to record the transaction. Transaction details include the order number, amount, and credit card information. Process 2.3 needs a real-time link to a credit bureau to get a credit authorization for the customer's credit card. This needs to be a

FIGURE B-14 A detailed diagram for Create new order (diagram 2)



real-time link rather than a data flow because data needs to flow back and forth rapidly while the process is executing. If the credit card is approved, a record of the transaction is created in the Order transaction data store and a data flow for the transaction goes directly to the bank.

The final process produces the order confirmation for the customer and the order details that go to shipping. Using the order number, process 2.4 looks up data on the Order, the Customer, and each Order item (plus the item description from the Product item) and then produces the required outputs.

■ Physical and Logical DFDs

physical DFD A DFD that includes one or more assumptions about implementation technology

logical DFD a DFD developed under the assumption of perfect internal technology

perfect internal technology an assumption that includes such technology capabilities as instant processing and data retrieval, infinite storage and network capacity, and a complete absence of errors

A DFD can be a physical system model, a logical system model, or a blend of the two. In a **physical DFD**, one or more assumptions about implementation technology will be embedded in the DFD. These assumptions can take many forms and might be very difficult to spot. A **logical DFD** is drawn as if the system might be implemented with any technology. In fact, one way that analysts develop logical DFDs is to assume that the system is implemented with **perfect internal technology**. Specifics of this assumption include process that never make mistakes, data stores that can hold infinite amounts of data without ever losing them, and data flows and processes with infinite capacity and zero execution or transmission time. The analyst simply omits any system feature that wouldn't exist under the perfect internal technology assumption.

Consider whether diagram 2 in Figure B-14 is a logical model. First, is it clear what type of computer is doing the processing? Could it be a desktop system?

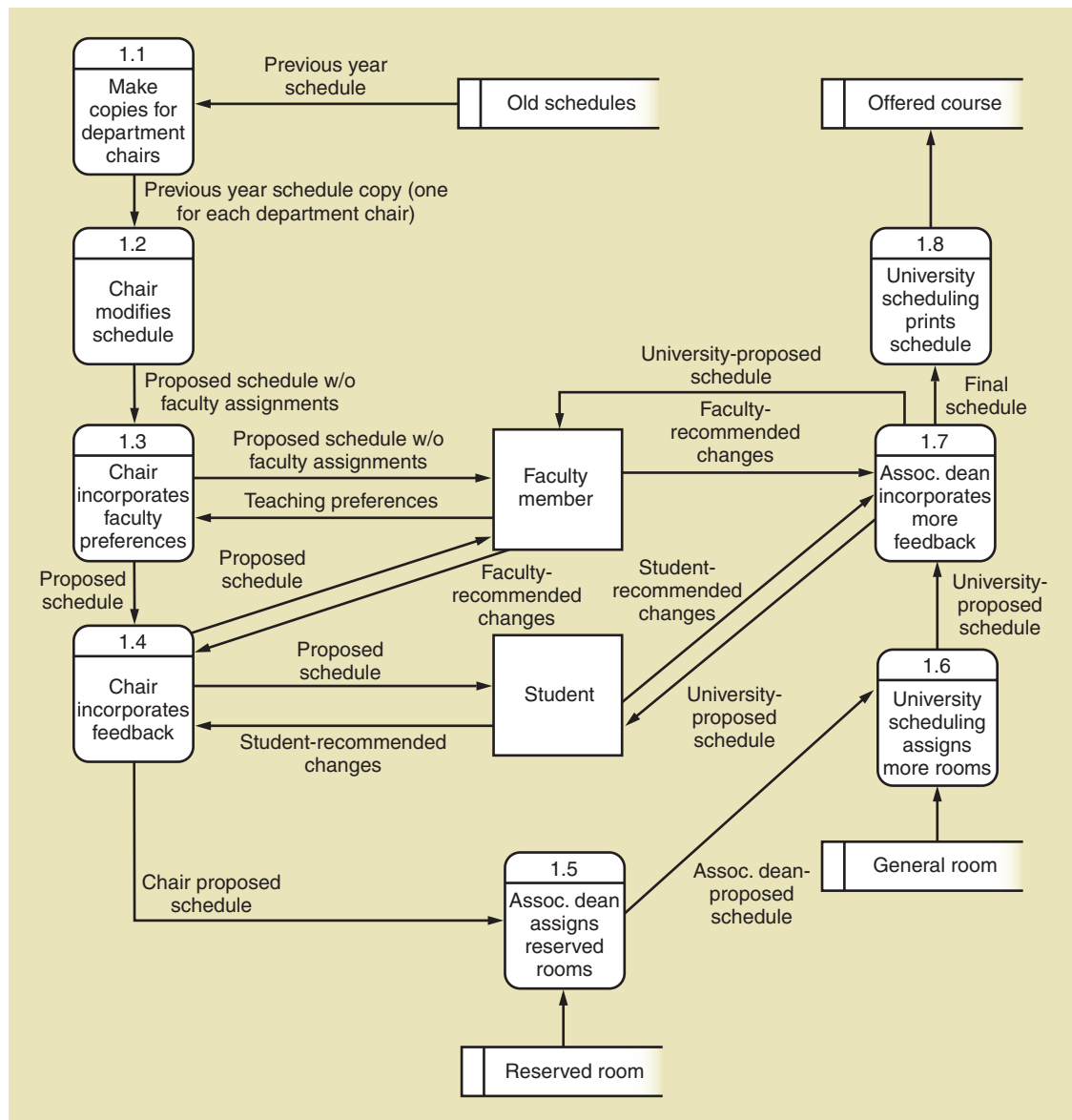
A tablet or phone app? A cloud computing solution? Or could the entire process as just described be carried out by people without any computer at all? Similarly, are the data stores sequential computer files? Are they tables in a relational database? Or are they files of paper in a file cabinet? How does the system get the data flow New order from the customer so it can be processed? By clicking check boxes and list boxes in a Windows application? Or on a Web page? Or by manually filling out a form that is later scanned into the system? Or by talking to the clerk over the phone? Or by talking to a speech recognition program over the phone?

All the alternatives described are possible, and if the model is a logical model, you shouldn't be able to tell how the system is implemented. At the same time, the processing requirements (what must go on) should be fairly detailed—down to indicating what attribute values are needed. The model could be even more detailed and still be a logical model.

Now consider whether the DFD in **Figure B-15** is a physical system model by comparing it with diagram 1 in Figure B-6. A number of elements indicate assumptions about implementation technology, including:

- Technology-specific processes
- Actor-specific process names

FIGURE B-15 A physical DFD for scheduling courses



- Technology- or actor-specific process orders
- Redundant processes, data flows, and files

The most obvious technology assumption is embedded in the name of process 1.1. Making copies is an inherently manual task, which implies that the data store Old schedules and the data flows into and out of process 1.1 are paper. It is possible that the data store and flows are electronic, but if so, the question arises why a process would be needed to make electronic copies.

Many of the process names include actors in the system. References to Chair, Assoc. dean, and University scheduling all indicate that a particular individual or department performs a process. The sequential flow of data among the processes is a by-product of the person or department that carries out each process. One can imagine alternate implementations with fewer processes, different process orders, or different assignment of processes to individuals and departments. The DFD clearly models one very specific set of decisions about process ordering and responsibility.

The DFD also includes processes with similar or redundant processing logic. For example, faculty input is accepted early, but faculty members later perform error checking twice (the data flows from processes 1.4 and 1.7). Also, rooms are assigned at two different times from two different data stores (Reserved room for process 1.5 and General room for process 1.6). As before, these features indicate very specific assumptions about the technology and division of responsibility. The redundant error checking indicates that it is possible for previous processes to make mistakes. A system implemented with perfect internal technology needs no internal error checking. The partitioning of room assignment between two files and processes may be related to technology (for example, no single process could successfully assign all rooms at once) or it could indicate a historic division of responsibility for room assignment.

Inexperienced analysts often develop DFDs, such as the one in Figure B-15. The path to developing such a model is simple: Model everything the current system does exactly the way it does it. The problem with this approach is that design assumptions based on outdated technology limitations can become inadvertently embedded in the new system. This problem is most prevalent when analysis and design are performed by different persons or teams. The designer(s) may not realize that some of the “requirements” embedded in the DFDs are simply reflections of the way things were in the past, not the way they necessarily should be in the future.

Physical DFDs are sometimes developed and used during the last stages of analysis or early stages of design. They are useful models for describing alternate implementations of a system prior to developing more detailed design models. But analysts should avoid creating physical DFDs during all analysis activities, except when generating design and implementation alternatives. Even during that activity, analysts should clearly label physical DFDs as such so readers know the model represents one possible implementation of the logical system requirements.

■ Evaluating DFD Quality

A high-quality set of DFDs is readable, is internally consistent, and accurately represents system requirements. Accuracy of representation is determined primarily by consulting users and other knowledgeable stakeholders. A project team can ensure readability and internal consistency by applying a few simple rules to DFD construction. Analysts can apply these rules while developing the DFDs or during a separate quality check after preparing DFD drafts.

■ Minimizing Complexity

People have a limited ability to manipulate complex information. If too much information is presented at once, people experience a phenomenon called

information overload difficulty in understanding that occurs when a reader receives too much information at one time

rule of 7 ± 2 (Miller's number) the rule of model design that limits the number of model components or connections among components to no more than nine

minimization of interfaces a principle of model design that seeks simplicity by limiting the number of connections among model components

information overload. When information overload occurs, a person has difficulty in understanding. The key to avoiding information overload is to divide information into small and relatively independent subsets. Each subset should contain a comprehensible amount of information that people can examine and understand in isolation.

A layered set of DFDs is an example of dividing a large set of information into small, independent subsets. Each DFD can be examined in isolation. The reader can find additional detail about a specific process by moving down to the next level or find information about how a DFD relates to other DFDs by examining the next-higher-level DFD.

An analyst can avoid information overload within any single DFD by following two simple rules of DFD construction:

- 7 ± 2
- Interface minimization

The **rule of 7 ± 2** (also known as **Miller's number**) derives from psychology research, which shows that the number of information “chunks” that a person can remember and manipulate at one time varies between five and nine. A larger number of chunks causes information overload. Information chunks can be many things, including names, words in a list, digits, or components of a picture.

Some applications of the rule of 7 ± 2 to DFDs include:

- A single DFD should have no more than 7 ± 2 processes.
- No more than 7 ± 2 data flows should enter or leave a process, data store, or data element on a single DFD.

These rules are general guidelines, not unbreakable laws. DFDs that violate these rules may still be readable, but violations should be considered a warning of potential problems.

Minimization of interfaces is directly related to the rule of 7 ± 2 . An interface is a connection to some other part of a problem or description. As with information chunks, the number of connections that a person can remember and manipulate is limited, so the number of connections should be kept to a minimum. Processes on a DFD represent chunks of business or processing logic. They are related to other processes, entities, and data stores by data flows. A single process with a large number of interfaces (data flows) may be too complex to understand. This complexity may show up directly on a process decomposition as a violation of the rule of 7 ± 2 . An analyst can usually correct the problem by dividing the process into two or more subprocesses, each of which should have fewer interfaces.

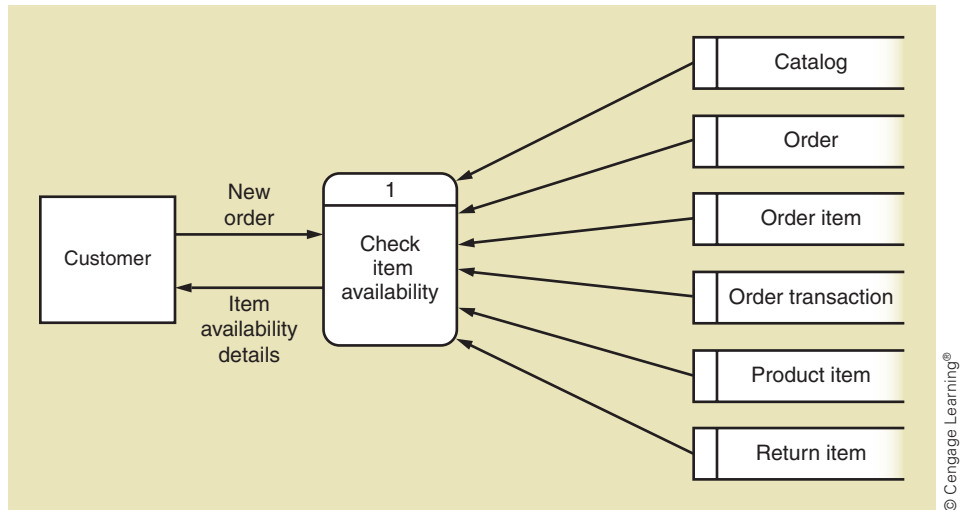
Pairs or groups of processes with a large number of data flows between them are another violation of the interface minimization rule. Such a condition usually indicates a poor partitioning of processing tasks among the processes. The way to fix the problem is to reallocate the embedded tasks so fewer interfaces are required. The best division of work among processes is the simplest—and the simplest division is one that requires the fewest interfaces among processes.

■ Ensuring Data Flow Consistency

An analyst can often detect errors and omissions in a set of DFDs by looking for specific types of inconsistency. Three common and easily identifiable consistency errors are:

- Differences in data flow content between a process and its process decomposition
- Data outflows without corresponding data inflows
- Data inflows without corresponding outflows

FIGURE B-16 A process with unnecessary data input—a black hole



balancing equivalence of data content between data flows entering and leaving a process and data flows entering and leaving a process decomposition DFD

A process decomposition shows the internal details of a higher-level process in a more detailed form. In most cases, the data content of flows to and from a process at one DFD level should be equivalent to the content of data flows to and from all processes in a decomposition. This equivalency is called **balancing**, and the higher-level DFD and the process decomposition DFD are said to be “in balance.”

Note the use of the term *data content* in the previous paragraph. Data flow names can vary among DFD levels for a number of reasons, including the decomposition of one combined data flow into several smaller flows. Thus, the analyst must be careful to look at the components of data flows, not just data flow names. For this reason, detailed analysis of balancing shouldn’t be undertaken until data flows have been fully defined.

Unbalanced DFDs may be acceptable when the imbalance is due to data flows that were ignored at the higher levels. For example, diagram 0 for a large system usually ignores details of error handling, such as when an item is ordered but is later determined to be out of stock and discontinued by its manufacturer. A process called Fulfill order in diagram 0 wouldn’t have any data flows associated with this condition. In the process decomposition of Fulfill order, the analyst might add a process and data flows to handle discontinued items.

Another type of DFD inconsistency can occur between the data inflows and outflows of a single process or data store. By definition, a process transforms data inflows into data outflows. In a logical DFD, data shouldn’t be needlessly passed into a process. The following consistency rules can be derived from these facts:

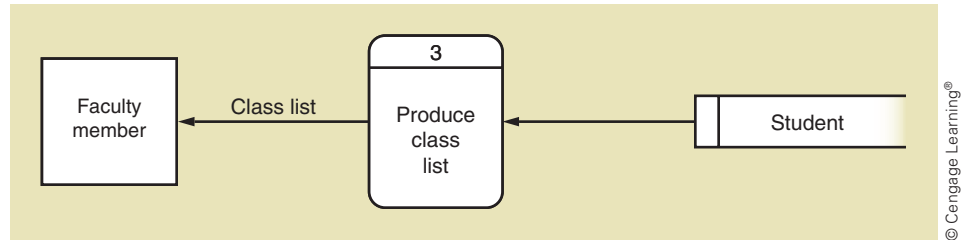
- All data that flows into a process must flow out of the process or be used to generate data that flows out of the process.
- All data that flows out of a process must have flowed into the process or have been generated from data that flowed into the process.

Figure B-16 shows an example that violates the first rule. Compare Figure B-16 with the first DFD fragment in Figure B-12, and note the difference in the data inflows to the process. Looking up item availability requires only information to identify the item and access to corresponding data stores. In Figure B-16, excess data input (an entire order) flows into the process, and the process accesses more data stores than needed to generate the data outflow Item availability details. A process such as the one shown in Figure B-16 is sometimes called a **black hole** because some or all of the data that enters never leaves. A more obvious black hole example is any process with at least one data inflow but no data outflows.

Figure B-17 shows an example that violates the second rule. Compare Figure B-17 with the bottom DFD fragment in Figure B-7, and note the difference

black hole a process or data store with a data input that is never used to produce a data output

FIGURE B-17 A process with an impossible data output—a miracle



miracle a process or data store with a data element that is created out of nothing

in the data inflows to the process. In Figure B-17, insufficient data enters the process to produce the data output. Required data inputs from the Offered course and Course enrollment are missing. A process such as the one shown in Figure B-17 is sometimes called a **miracle** because data emerges from the process without any apparent source.

Analysts sometimes can spot black holes and miracles simply by examining the DFD. In other cases, close examination of the data dictionary or process descriptions is required. In Figure B-18, data elements A, B, and C flow into the process but don't flow out. Data element A is used to determine what formula to apply to recompute the value of X, so data element A is a necessary input. However, data elements B and C play no role in generating process output and thus should be eliminated as unnecessary inflows.

In Figure B-19, data elements A, B, and Y flow out of the process. Data element A flows into the process. Data element Y is computed by an algorithm

FIGURE B-18 A process with unnecessary data input

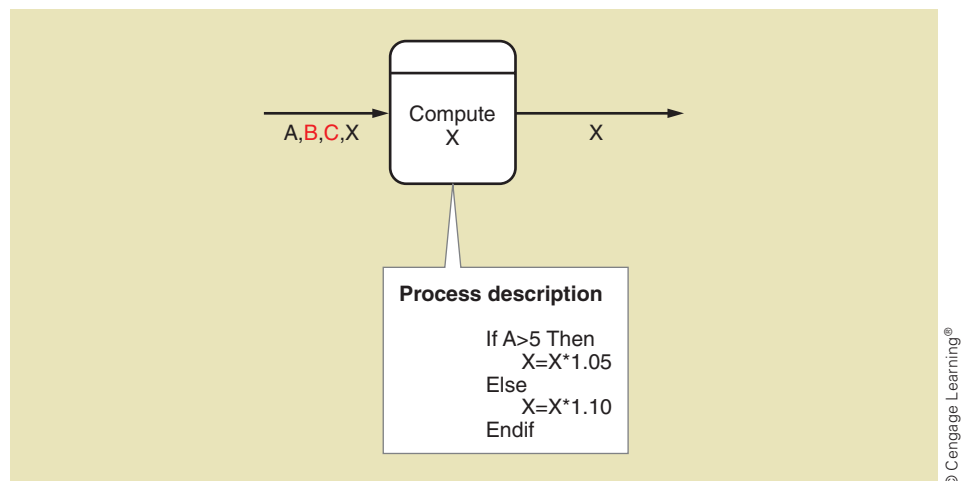
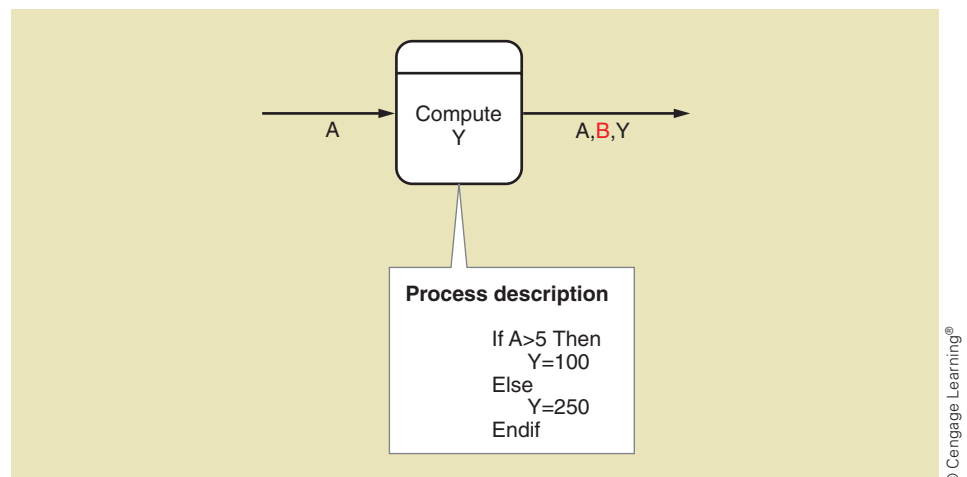


FIGURE B-19 A process with an impossible data output



based on data element A. However, data element B doesn't flow into the process and isn't computed by internal processing logic. Thus, data element B indicates either an error in the data outflow (B should be eliminated) or an omission in the internal processing logic (the rule that determines B is missing).

Note that both consistency rules apply to data stores as well as processes. Any data element that is read from a data store must have been previously written to that data store. Similarly, any data element that is written to a data store must eventually be read from the data store. Examining the consistency of data flows to and from a data store is complicated by the fact that a data element may flow into and out of a data store on completely different DFDs.

Evaluating data flow consistency is a straightforward but tedious process. Fortunately, most analysis modeling tools automatically perform data flow consistency checking. But those tools place rigorous requirements on the analyst to precisely specify the internal logic of processes. Without precise process descriptions, it is impossible for the tool (or a human being) to know what data elements are used as input or generated as output by internal processing logic.

■ Documentation of DFD Components

Data flow diagrams graphically summarize interactions among three types of internal system components—processes, data flows, and data stores—but additional details about each component need to be described. First, each lowest-level process needs to be described in detail. In addition, the analyst needs to define each data flow in terms of the data elements it contains. Data stores also need to be defined in terms of the data elements. Finally, the analyst also needs to define each data element.

■ Process Descriptions

Each process on a DFD must be defined formally. There are several options for process definition, including one that has already been discussed: process decomposition. As discussed previously, in a process decomposition, a higher-level process is formally defined by a DFD that contains lower-level processes. These lower-level processes may in turn be further decomposed into even lower-level DFDs.

Eventually, a point is reached at which a process doesn't need to be defined further by a DFD. This point occurs when a process becomes so simple that it can be described adequately by other methods: structured English, decision tables, or decision trees. With each method, the process is described as an algorithm and an analyst chooses the most appropriate presentation format by determining which is most compact, readable, and unambiguous. In most cases, structured English is the preferred method.

structured English a method of writing process specifications that combines structured programming techniques with narrative English

Structured English uses brief statements to describe a process very carefully. Structured English looks a bit like programming statements but without references to computer concepts. Rules of structured programming are followed, and indentation is used for clarity. For example, a simple set of instructions for processing ballots after a vote is shown in **Figure B-20**. Some statements are simply instructions. Other statements repeat instructions. Still other statements direct the program to execute one set of instructions or the other. The procedure always starts at the top and ends at the bottom. Therefore, the rules of structured programming apply. Note, though, that a process described by structured English isn't necessarily a computer program—it might be done by a person—so it is a logical model. It is unambiguous, so anyone following the instructions will arrive at the same result.

An example of a process description for the RMO CSS is shown in **Figure B-21**. Note how the process description provides more specific details

FIGURE B-20 A structured English example

Process Ballots Procedure

```

Collect all ballots
Place all ballots in a stack
Set Yes count and No count to zero
Repeat for each ballot in the stack
    If Yes is checked then
        Add one to Yes count
    Else
        Add one to No count
    Endif
    Place ballot on counted ballot stack
Endrepeat
If Yes count is greater than No count then
    Declare Yes the winner
Else
    Declare No the winner
Endif
Store the counted ballot stack in a safe place
End Process Ballots Procedure

```

© Cengage Learning®

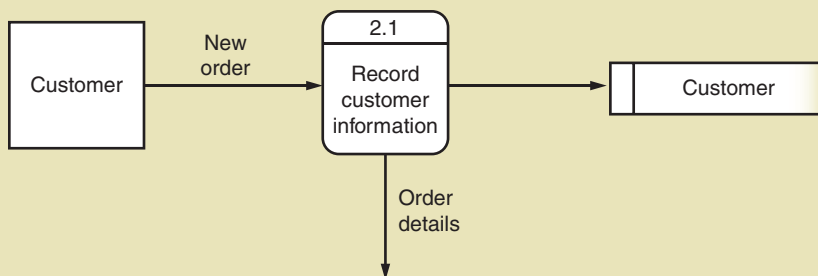
FIGURE B-21 RMO CSS process 2.1 (Record customer information) and its structured English process description

Process 2.1 - Record Customer Information

```

Ask if customer has an account (or has made a previous order)
If customer has an account then
    Ask for identification information
    Query database with identifying information
    Copy query response data to Order details
Else
    Create an empty Customer record in the database
    Ask customer for Customer attributes
    Update empty Customer record with Customer attributes
Endif
Ask customer for order information for first item
While more order items Do
    Update Order details with order information
Endwhile

```



© Cengage Learning®

about what the process does. If one process description method becomes too complex, the analyst should choose another. Excess length (for example, more than 20 lines) or multiple levels of indentation (indicating complex decision logic) indicate that a structured English description may be too complex. An analyst can sometimes address excess indentation by converting the description to an equivalent decision table or decision tree. In other cases, a process decomposition may be required.

Structured English is well suited to describing processes with many sequential processing steps and relatively simple control logic (such as a single loop or an if-then-else statement). Structured English isn't well suited for describing processes with the following characteristics:

- Complex decision logic
- Few (or no) sequential processing steps

Decision logic is complex when multiple decision variables and a large number of possible combinations of those variables need to be considered. When a process with complex decision logic is described with structured English, the result is typically a long and difficult-to-read description. For example, consider the structured English description for calculating shipping costs shown in **Figure B-22**. Note that the description is relatively long and consists mostly of control structures (if, else, and endif statements).

Decision tables and **decision trees** can summarize complex decision logic more concisely than structured English. **Figures B-23** and **B-24** show a decision table and decision tree that represent the same logic as the structured

decision table a tabular representation of processing logic containing decision variables, decision variable values, and actions or formulas

decision tree a graphical description of process logic that uses lines organized like branches of a tree

FIGURE B-22 A structured English process description for determining delivery charges

```

If YTD purchases > $250 then
  If number of items ordered < 4 then
    If delivery date is next day then
      delivery charge is $25
    Endif
    If delivery date is second day then
      delivery charge is $10
    Endif
    If delivery date is seventh day then
      delivery charge is $1.50 per item
    Endif
  Else
    If delivery date is next day then
      delivery charge is $6 per item
    Endif
    If delivery date is second day then
      delivery charge is $2.50 per item
    Endif
    If delivery date is seventh day then
      delivery charge is zero (free)
    Endif
  Endif
Else
  If number of items ordered < 4 then
    If delivery date is next day then
      delivery charge is $35
    Endif
    If delivery date is second day then
      delivery charge is $15
    Endif
    If delivery date is seventh day then
      delivery charge is $10
    Endif
  Else
    If delivery date is next day then
      delivery charge is $7.50 per item
    Endif
    If delivery date is second day then
      delivery charge is $3.50 per item
    Endif
    If delivery date is seventh day then
      delivery charge is $2.50 per item
    Endif
  Endif
Endif

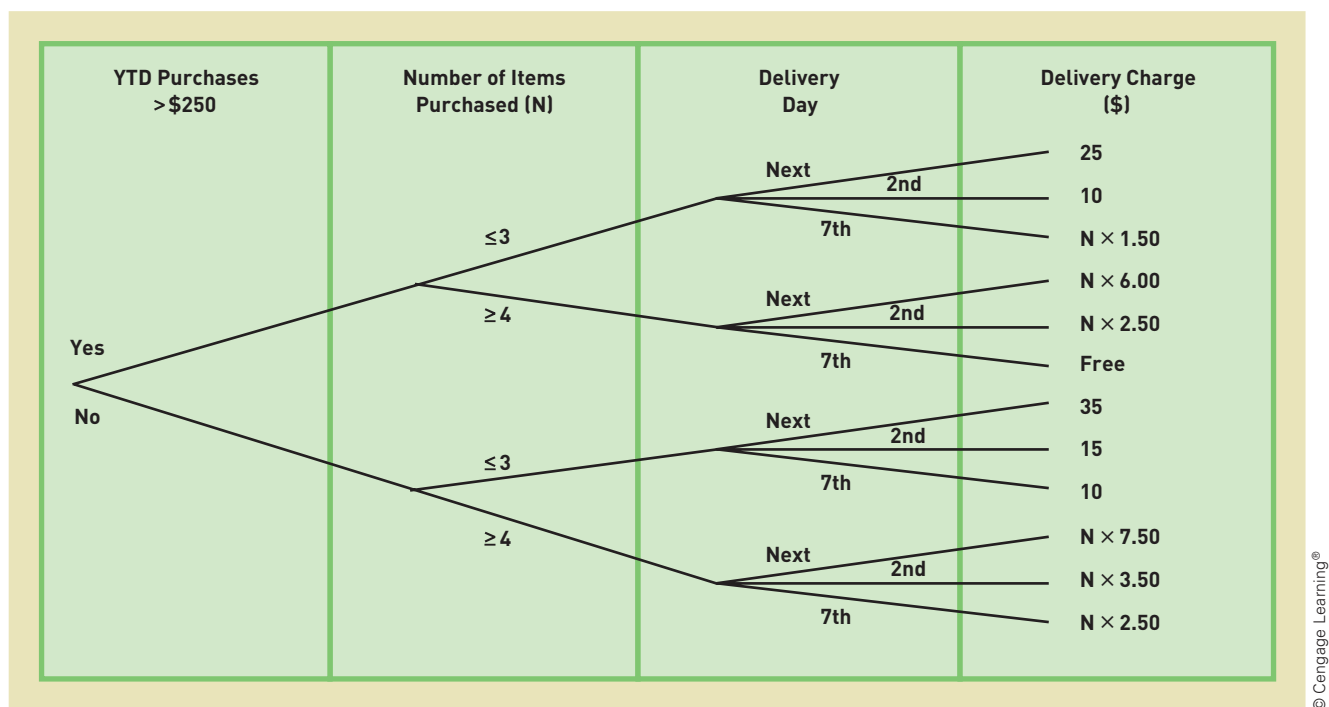
```

FIGURE B-23 A decision table for calculating shipping charges

YTD Purchases > \$250	YES						NO					
Number of Items (N)	N ≤ 3			N ≥ 4			N ≤ 3			N ≥ 4		
Delivery Day	Next	2nd	7th	Next	2nd	7th	Next	2nd	7th	Next	2nd	7th
Shipping Charge (\$)	25	10	$N \times 1.50$	$N \times 6.00$	$N \times 2.50$	Free	35	15	10	$N \times 7.50$	$N \times 3.50$	$N \times 2.50$

© Cengage Learning®

FIGURE B-24 A decision tree for calculating shipping charges



© Cengage Learning®

English example in Figure B-22. Both incorporate decision logic into the structure of the table or tree to make the descriptions more readable than their structured English equivalent. The decision table is more compact, but the decision tree is easier to read. Sometimes, an analyst needs to describe a process all three ways before deciding which approach describes a particular process best.

The following steps are used to construct a decision table:

1. Identify each decision variable and its allowable values (or value ranges).
2. Compute the number of decision variable combinations as the product of the number of values (or value ranges) of each decision variable.
3. Construct a table with one more column than the number of decision variable combinations computed in step 2 (the extra column is for decision variable names and process action or computation descriptions). The table should have a row for each decision variable and a row for each process action or computation.

4. Assign the decision variable with the fewest values (or value ranges) to the first row of the table. Put the decision variable name in the first column. Divide the remaining columns into sets of columns for each decision variable value (or value range).
5. Choose the next decision variable with the fewest values (or value ranges) for the second row. Put the variable's name in the first column. Compute the number of column groups as the product of the number of values (or value ranges) of this variable and all the variables above it in the table. Divide the remaining columns into the computed number of groups, and insert values (or value ranges) in a regular pattern.
6. Continue inserting rows as instructed in step 5 until all decision variables have been included in the table.
7. Add a row for each calculation or action. For each calculation cell, insert the appropriate constant value or formula for the combination of decision variable values that appear above the cell in the same column. For each action cell, place a check mark in the cell if that action is performed when the decision variables have the values shown in the column above the cell.

Now let us follow these steps to show how the decision table in Figure B-23 was constructed. There are three decision variables: year to date (YTD) purchases, number of items ordered, and delivery day. YTD purchases has two relevant ranges: less than \$250 and greater than or equal to \$250. Note that decision variable ranges must be mutually exclusive and collectively exhaustive. Number of items ordered also has two relevant ranges: less than or equal to three and greater than or equal to four. Delivery day has three possible values: next day, second day, and seventh day. There are $2 \times 2 \times 3 = 12$ combinations of values, so there are 13 columns in the table to allow for a decision variable name, the formula, and the action names.

YTD purchases and the number of items have two relevant value ranges, so either can occupy the first row. We chose YTD purchases. It has two value ranges, so we created two groups of $12 \div 2 = 6$ columns and labeled them for each possible value. The next row is for number of items. It has two ranges, so we need four groups of three columns—that is, $12 \div 2$ value ranges for the number of items $\div 2$ value ranges for YTD purchases. We insert the value ranges for the number of items into the column groups in a regular pattern, as shown in the sample figure. The delivery day is now inserted into the table. Because it is the last decision variable, we don't need to group any columns beneath it. We simply insert the values of the delivery date into individual columns in a regular pattern, as we did for the other decision variables.

The final step is to insert the row containing formulas and values for the shipping charges. Each cell contains a value or formula for the combination of decision variable values in the columns above. For example, shipping is free for customers with YTD purchases greater than \$250, orders of more than three items, and seventh-day delivery. The shipping charge is \$35 for customers with YTD purchases less than \$250, an order of three items or fewer, and next-day delivery.

If the decision table is used to represent a process that implements one or more actions—instead of value calculations, as in the previous example—then the table must contain a row for each action. Cells in these rows are check-marked to indicate which actions are performed under which conditions. **Figure B-25** shows a simple example of this type of table. Two action rows are included, and the action is performed if a check mark appears in the cell immediately below the decision variable values. For example, if the customer is new and the shipment contains an item back-ordered for more than 25 days, then the shipment is expedited and the detailed return instructions are included in

FIGURE B-25 A simple decision table with multiple action rows

New customer	Yes		No	
Item back order $\geq$ 25 days	Yes	No	Yes	No
Include detailed return instructions	✓	✓		
Expedite delivery	✓		✓	

the container. If the customer isn’t new and the order contains no items back-ordered for more than 25 days, then neither action is taken.

You can construct a decision tree by using almost the same steps as listed previously for constructing a decision table. The primary difference is that rows in a decision table are columns in a decision tree and vice versa. To see this for yourself, draw an imaginary line through the table in Figure B-23 from the top left to the bottom right. Then, flip the table along the imaginary line and compare the structure of the flipped table to the decision tree in Figure B-24. The only other significant difference between a table and a tree is that a tree uses labeled branches instead of grouped columns to represent decision variable values.

■ Data Flow Definitions

data flow definition a textual description of a data flow’s content and internal structure

A data flow is a collection of data elements, so **data flow definitions** list all the elements. For example, a simplified New order data flow (to process 2.1 in Figure B-14) consists of a customer name, credit card number, and list of catalog item numbers and quantities. Some of these elements are actually structures of other elements, such as a customer name consisting of first name, middle initial, and last name. The system stores most of these data elements, so they coincide with the attributes of data entities included in the ERD.

Sometimes, data flow definitions contain a more complex structure. In the New order example, each data flow consists of many catalog items and quantities (a repeating group). It is important to document this structure. The notations for data flow definitions vary. One approach is simply to list the data elements, as shown in Figure B-26. The elements that can have many values are indicated. Another approach uses an algebraic notation, such as that shown in Figure B-27. The data flow “equals” or “consists of” one element plus another element and so on. Groups of elements that can have many values are enclosed in curly braces. This example shows New order “equals” the customer name

FIGURE B-26 Data flow definitions simply listing elements

Customer-Name
Customer-Address
Credit-Card-Information
Item-Number
Quantity

FIGURE B-27 Algebraic notation for data flow definition (New-Order)

New-Order = Customer-Name + Customer-Address +
Credit-Card-Information + { Item-Number + Quantity }

“plus” customer address “plus” credit card information “plus” “one or more” inventory item number and quantity. In this example, the customer name can be defined separately as a structure of elements.

Figures B-28 and B-29 show a complex report and its corresponding data flow definition. The structure of the report is a repeating group of products with an embedded repeating group of inventory items. The data flow definition

FIGURE B-28 A sample report produced by the RMO customer support system



Ridgeline Mountain Outfitters—Products and Items

ID	Name	Season	Category	Supplier	Unit Price	Special	Special Price	Discontinued
RM0125	Outdoor Field Spr/Fall Mens C		8201		\$39.00		\$0.00	No
Description Outdoor Nylon Jacket with Lining								
	Size	Color	Style	Units in Stock	Reorder Level	Units on Order		
	Large	Blue		1500	150			
	Large	Green		1500	150			
	Large	Red		1500	150			
	Large	Yellow		1500	150			
	Medium	Blue		1500	150			
	Medium	Green		1500	150			
	Medium	Red		1500	150			
	Medium	Yellow		1500	150			
	Small	Blue		1500	150			
	Small	Green		1500	150			
	Small	Red		1500	150			
	Small	Yellow		1500	150			
	Xlarge	Blue		1500	150			
	Xlarge	Green		1500	150			
	Xlarge	Red		1500	150			
	Xlarge	Yellow		1500	150			

ID	Name	Season	Category	Supplier	Unit Price	Special	Special Price	Discontinued
RM0125	Hiking Walkers All		Footwear	7993	\$49.95		\$0.00	No
Description Hiking Walkers with Patterned Tread Durable Uppers								
	Size	Color	Style	Units in Stock	Reorder Level	Units on Order		
	10	Brown		1000	100			
	10	Tan		1000	100			
	11	Brown		1000	100			
	11	Tan		1000	100			
	12	Brown		1000	100			
	12	Tan		1000	100			
	13	Brown		1000	100			
	13	Tan		1000	100			
	7	Brown		1000	100			
	7	Tan		1000	100			
	8	Brown		1000	100			
	8	Tan		1000	100			
	9	Brown		1000	100			
	9	Tan		1000	100			

FIGURE B-29 A data flow definition for the RMO products and items report

```
products-and-items-report =
  {
    {
      product-id + product-name + season + category +
      supplier + unit-price + special + special-price +
      discontinued + description +
      {
        size + color + style + units-in-stock +
        reorder-level + units-on-order
      }
    }
  }
```

© Cengage Learning®

captures this structure by embedding the item repeating group within one set of curly braces and the product repeating group within the outermost set of curly braces.

■ Data Store Definitions

Because a data store on the DFD represents a data entity on the ERD, no separate definition is typically needed (except perhaps a note referring the reader to the ERD). If data stores aren't linked to an ERD, the analyst simply defines the data store as a collection of elements (possibly with a structure) in the same way data flows are defined.

■ Data Element Definitions

Data element definitions describe a data type, such as string, integer, floating point, or Boolean. Each element should also be described to indicate specifically what it represents. Sometimes these descriptions are very specific. A date of sale might be defined as the date the payment for the order was received. Alternately, the date of sale might be the date an order is placed. Sometimes, different departments in the same company have different definitions for the same element, so it is very important for the analyst to confirm exactly what the element means to users.

Other parts of a data element definition vary depending on the type of data. A length is usually defined for a string. For example, a middle initial might be one character maximum, but how long should a first name be? Numeric values usually have a minimum and maximum value that can be defined as a valid range. Sometimes, specific values are allowed for the element, such as valid codes. If the element is a code, it is important to define the valid codes and their meaning. For example, code A might mean ship immediately, code B might mean hold for one day, and code C might mean hold shipment pending confirmation. Some sample data element definitions are shown in **Figure B-30**.

Analysts need to maintain a central store of all these definitions as a project reference and to ensure consistency. A **data dictionary** is a repository for definitions of data flows, data stores, and data elements. A data dictionary may be a simple loose-leaf notebook or word processing file in smaller development projects. In larger projects, a project management or documentation tool usually holds the data dictionary. The data dictionary may also hold process descriptions.

data dictionary a repository for definitions of data flows, data elements, and data stores

■ DFD Summary

Figure B-31 shows each of the components of a traditional analysis model: an entity-relationship diagram, data flow diagrams, process definitions, and data definitions.

FIGURE B-30 *Data element definitions*

```

units-in-stock =
    a positive integer

supplier =
    a four digit numeric code

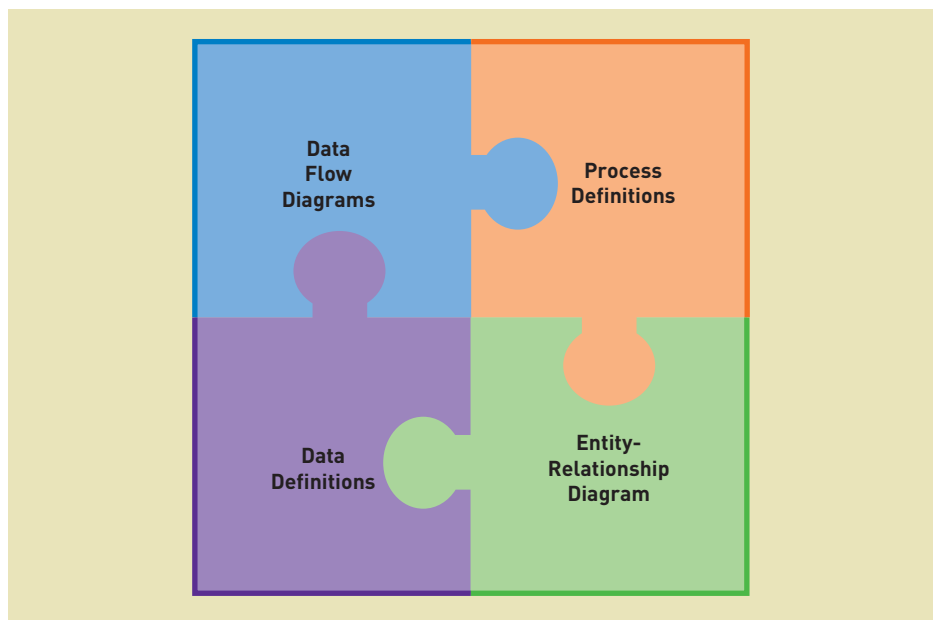
unit-price =
    a positive real number accurate to two decimal places,
    always in U.S. dollars

description =
    a text field containing a maximum of 50 printable characters

special =
    a coded field with one of the following values
    0: item is not "on special"
    1: item is "on special"

```

© Cengage Learning®

FIGURE B-31 *The components of a traditional systems analysis model*

© Cengage Learning®

The four components form an interlocking set of specifications for most system requirements. The data flow diagram provides the highest-level view of the system, summarizing processes, external agents, data stores, and the flow of data among them. Each of the other components describes some aspect of the data flow diagram in greater detail.

The models described thus far were developed in the 1970s and 1980s as part of the traditional structured analysis methodology (see Yourdon 1989 in the “Further Resources” section). They were designed to completely document the logical requirements of a system. However, some analysts chose to augment the structured models with models borrowed from other methodologies to describe information not captured by the structured models or to present similar information in a slightly different form. The remainder of this chapter describes some of these “borrowed” models and ways they can be used to augment the traditional structured analysis models.

■ Locations and Communication through Networks

Because structured systems analysis concentrates on logical modeling, physical issues such as processing locations and networks are sometimes ignored during analysis. However, a great deal of information about process, data, and user distribution is needed during the early stages of design. Gathering location information during analysis is often more efficient. Examples of location information include:

- The number of locations of users
- Processing and data access requirements of users at specific locations
- Volume and timing of processing and data access requests

The first step in gathering location information is to identify and describe the locations where work is being done or where it will be performed. Possible locations include business offices, warehouses, and manufacturing facilities as well as such less obvious locations as customer or supplier offices, employee homes, hotel rooms, and automobiles. All these locations should be listed, and a **location diagram** should be drawn to summarize the locations graphically. A location diagram for the current RMO CSS is shown in **Figure B-32**. The location diagram is often used as a starting point for defining network communication requirements.

location diagram a diagram or map that identifies all the processing locations of a system

FIGURE B-32 The RMO location diagram

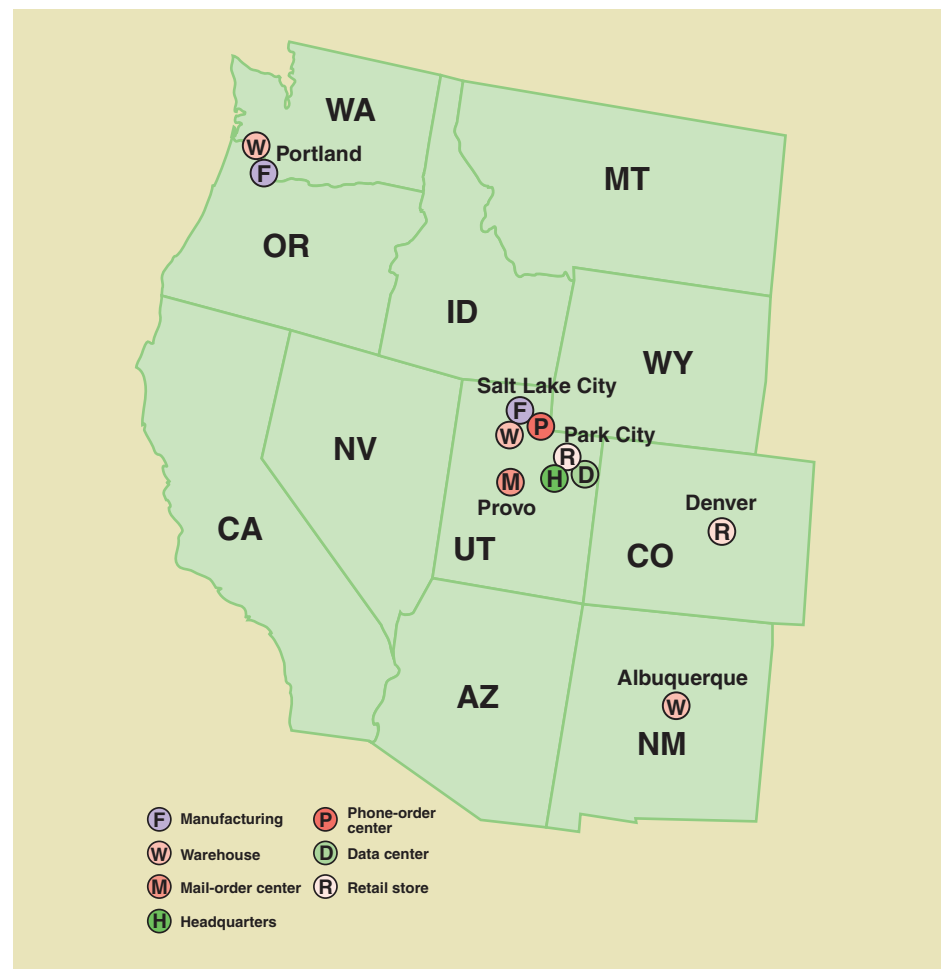


FIGURE B-33 Activity-location matrix for the RMO customer support system

ACTIVITY	LOCATION				
	Corporate Offices (Park City)	Distribution Warehouses (Salt Lake City, Albuquerque, Portland)	Mail-Order (Provo)	Phone Sales (Salt Lake City)	Customer Direct Interaction (Anticipated)
Look up item availability	X	X	X	X	X
Create new order			X	X	X
Update order				X	X
Look up order status	X	X		X	X
Record order fulfillment		X			
Record back order		X			
Create order return		X			
Provide catalog info			X	X	X
Update customer account	X		X	X	X
Distribute promotional package	X				
Create customer charge adjustment	X				
Update catalog	X				
Create special product promotion	X				
Create new catalog	X				

© Cengage Learning®

activity-location matrix a table that describes the relationship between processes and the locations in which they are performed

activity-data matrix a table that describes stored data entities, the locations from which they are accessed, and the nature of the accesses

CRUD acronym for create, read, update, and delete

The next step is to list the functions that are performed by users at each location. Using the event table, the analyst can list where each activity is performed. **Figure B-33** shows an **activity-location matrix** that summarizes this information. Each row is a system activity, and each column represents a location. Many activities are performed at multiple locations.

Other matrices can be created to highlight access requirements. One approach is to list activities and data entities (or classes of objects) in an **activity-data matrix**. This matrix shows which activities require access to the data or objects. This information can be found on the DFD fragments for the traditional approach and on the sequence diagrams for the object-oriented approach. In either approach, creating a matrix to summarize this information can be useful.

Figure B-34 shows an activity-data matrix for the RMO CSS. The cells of the matrix show additional information to clarify what the activity does to the data. The letter C means the activity creates new data, R means the activity reads data, U means the activity updates data, and D means the activity might delete data. The acronym **CRUD** (create, read, update, and delete) is often used to describe this type of matrix.

FIGURE B-34 *RMO activity-data matrix*

ACTIVITIES	DATA ENTITIES										
	Catalog	Customer	Inventory Item	Order	Order Item	Order Transaction	Package	Product Item	Return Item	Shipment	Shipper
Look up item availability			R								
Create new order		CRU	RU	C	C	C	R	R		C	R
Update order		RU	RU	RUD	RUD	RUD	R	R		CRUD	R
Look up order status		R		R	R	R				R	R
Record order fulfillment					RU					RU	
Record back order					RU					CRU	
Create order return		CRU		RU		C			C		
Provide catalog info	R		R				R	R			
Update customer account		CRUD									
Distribute promotional package	R	R	R				R	R			
Create customer charge adjustment		RU				CRUD					
Update catalog	RU		R				RU	R			
Create special product promotion	R		R				R	R			
Create new catalog	C		R				CRU	R			

C = Creates new data, R = Reads existing data, U = Updates existing data, D = Deletes existing data

CHAPTER SUMMARY

Data flow diagrams (DFDs) are used in combination with the use cases and entity-relationship diagram (ERD) to model system requirements. DFDs model a system as a set of processes, data flows, external agents, and data stores. DFDs are relatively easy to read because they graphically represent key features of the system by using a small set of symbols. Because there are many features to be represented, many types of DFDs are developed, including context diagrams, DFD fragments, subsystem DFDs, event-partitioned DFDs, and process decomposition DFDs.

Each process, data flow, and data store requires a detailed definition. Analysts may define processes in a number of ways, including a structured English process specification, a decision table, a decision tree, or a process decomposition DFD. Process decomposition

DFDs are used when internal process complexity is too great to allow the creation of a readable, one-page definition by any other means. Data flows are defined in terms of their component data elements and their internal structure. Data elements may be further defined in terms of their type and allowable content. Data stores correspond to entities on the ERD and thus require no additional definition.

The location diagram, activity-location matrix, and activity-data matrix describe important information about system locations. The location diagram summarizes geographic locations where the system is to be used. The activity-location matrix describes which processes are implemented at which locations. The activity-data matrix summarizes where and how each data store is used.

KEY TERMS

activity-data matrix	data store	logical DFD
activity-location matrix	decision table	minimization of interfaces
balancing	decision tree	miracle
black hole	DFD fragment	perfect internal technology
context diagram	event-partitioned system model (diagram 0)	physical DFD
CRUD	external agent	process
data dictionary	information overload	rule of 7 ± 2 (Miller's number)
data flow	levels of abstraction	structured English
data flow definition	location diagram	
data flow diagram (DFD)		

REVIEW QUESTIONS

1. List at least three different types of DFDs. What is each diagram type used to represent?
2. List the five component parts (symbols) of a DFD. Briefly describe what each symbol represents.
3. How does an analyst determine whether a person or organization should be represented on a DFD as an external agent or by one or more processes?
4. Processes on an event-partitioned DFD can be described by a detailed DFD or a process specification. How does an analyst determine which is the most appropriate form of description?
5. How are entities from the ERD represented on a DFD? How are relationships from the ERD represented on a DFD?
6. What assumption differentiates a physical from a logical DFD? What features may be present on a physical DFD that should never be present on a logical DFD?
7. What DFD characteristics does an analyst examine when evaluating DFD quality?
8. What is a black hole? What is a miracle? How can each be detected?
9. Why might an analyst describe a process with a decision table or tree instead of structured English?
10. What is an activity-location matrix? How is it related to DFDs?
11. What is an activity-data matrix? How is it related to DFDs and the ERD?

PROBLEMS AND EXERCISES

1. Assume you are preparing a DFD to describe the process of creating, approving, and closing a mortgage loan by a mortgage broker. Should the broker be represented as an external agent or by one or more processes? Why? What about the closing agent, the credit bureau, and the bank that issues the mortgage note?
2. Examine the course registration system described in Figure B-6. Are there any other processes that would be required to implement a fully functioning system? Hint: Black holes and miracles may indicate processing steps that were left out of the DFD.
3. Assume that the transaction summary report for the RMO CSS order-entry subsystem (see process 5 in Figure B-12) contains a listing of every order that was created during a date range entered by the user. The report title page contains the report name, the date range, and the date and time the report was prepared. For each order, the report lists the order number, order date, order total, and form of payment. Within each order, the report lists all order items and returns, including item number, quantity ordered (or returned), and price. Report totals include the sum of all order totals, average order total, average item price, and average return price. Write a data flow definition entry for the report and then write a process specification for the process that produces the report.
4. Create an activity-data (CRUD) matrix for the course registration system in Figure B-6.
5. Develop a physical DFD that models the process of grocery shopping—from the time you write down a shopping list until the time you store purchased groceries in your home. Construct your DFD as a linear sequence of processes. Now develop a logical DFD to describe the same scenario. Try to develop a diagram that is equally valid as a logical description of the way you currently buy groceries and as a logical description of ways you might buy groceries without ever leaving your home.
6. Consider the admissions requirements for a degree program, major, or concentration at your school. Look up the requirements in the school catalog and then rewrite them in structured English. Develop an equivalent decision table and/or decision tree. Which is easier to understand? Why?
7. Get a copy of your school transcript. Write a data definition that describes its contents. Write data element definitions for the fields Grade, Credits, and Degree.
8. Define process 2 in Figure B-7 as it is implemented at your school. Use whatever combination of process decomposition and process specification is appropriate. If you develop any process decomposition DFDs, be sure to define all data flows.

CASE STUDY

Sandia Medical Devices

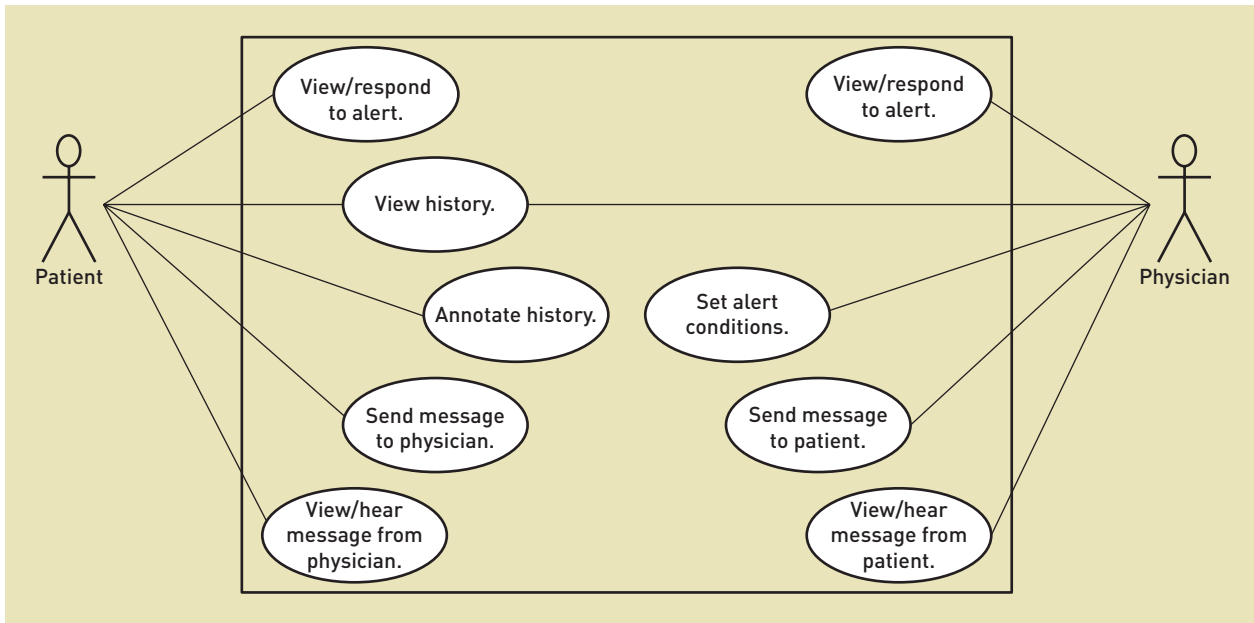
Review the specifics of the Sandia Medical Devices case in Chapters 2–5, including the figures reproduced here. Perform these tasks:

1. Develop a context diagram encompassing all the use cases in **Figure B-35**.
2. Develop an event-partitioned system model encompassing all the use cases in Figure B-35. Assume that the classes and associations in

Figure B-36 correspond to entities and relationships from a similar ERD.

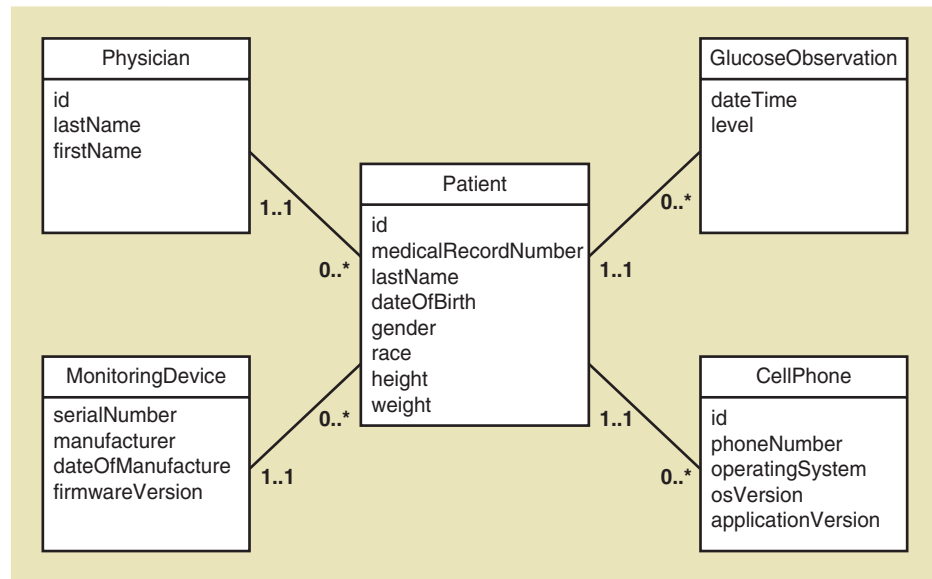
3. Develop a DFD fragment and a process explosion for the *View history* use case. Assume that the system will automatically display the most recent glucose level, which is updated at five-minute intervals by default. Assume further that the user can ask the system to view glucose levels during a user-specified time period and that the levels can be displayed in tabular form or as a graph.

FIGURE B-35 RTGM system use cases



© Cengage Learning®

FIGURE B-36 Initial domain model class diagram for Sandia RTGM system



© Cengage Learning®

FURTHER RESOURCES

- J. Martin, *Information Engineering: Book I: Introduction*. Prentice Hall, 1988.
- J. Martin, *Information Engineering: Book II: Planning and Analysis*. Prentice Hall, 1989.
- Stephen M. McMenamin and John F. Palmer, *Essential Systems Analysis*. Yourdon Press, 1984.
- G. A. Miller, "The magical number seven, plus or minus two: Some limits on our capacity for processing information." *Psychological Review*, volume 63 (1956), pp. 81–97.
- Edward Yourdon, *Modern Structured Analysis*. Yourdon Press, 1989.